

DeepAgent 实战：自己动手写一个 Hermes

高强文 编著

目 录

第 1 章 开发环境搭建与第一个程序	1
一、基本原理	1
1.1 它其实什么都不记得	1
1.2 Harness：模型外面那层工程	1
1.3 它有骨架，但没有外壳	2
1.4 全书路线	2
1.5 依赖环境：三样东西	3
二、程序介绍	3
2.1 完整程序	3
2.2 逐行讲解	4
2.3 这段程序里没有的东西	4
三、运行过程	4
3.1 环境准备	4
3.2 跑起来	5
四、本章小结	6
第 2 章 用 DeepAgent 开发最简智能体	7
一、基本原理	7
1.1 装进去的是一张图，不是模型	7
1.2 外壳不变，变的是中间层	7
1.3 包里带的七个文件工具	7
1.4 它进来时的两个默认	8

二、程序介绍	8
2.1 完整程序	8
2.2 与第 1 章的差别	9
2.3 那几行 print 是干什么的	9
2.4 两颗螺丝	9
三、运行过程	9
3.1 环境准备	9
3.2 第一次跑：把两颗螺丝都松开	10
3.3 加上第一颗螺丝：身份	10
3.4 加上第二颗螺丝：文件落盘	10
3.5 一次多步任务的实录	11
四、本章小结	12
第 3 章 工具调用：给智能体接入联网能力	13
一、基本原理	13
1.1 工具就是一个普通函数	13
1.2 docstring 是产品文案	13
1.3 搜索为什么自己造	14
1.4 返回的东西要进上下文	14
1.5 失败要能读懂	14
1.6 依赖环境	14
二、程序介绍	15
2.1 工具一：web_search	15
2.2 工具二：fetch_url	15

2.3 主程序.....	16
2.4 与第 2 章的差别.....	16
三、运行过程.....	17
3.1 环境准备.....	17
3.2 一次真实调研.....	17
3.3 实录里的三件事.....	18
3.4 现阶段的不足.....	18
四、本章小结.....	19
第 4 章 技能 (Skills)：把做事的方法固化成能力.....	20
一、基本原理.....	20
1.1 工具是手，技能是手艺.....	20
1.2 一个技能长什么样.....	20
1.3 三级渐进加载.....	21
1.4 技能是作者写的东西，只读.....	21
1.5 自进化与三道闸门.....	21
1.6 依赖环境.....	22
二、程序介绍.....	22
2.1 技能文件：SKILL.md.....	22
2.2 附件：references/report-template.md.....	23
2.3 主程序.....	23
2.4 三处改动各解决什么问题.....	24
三、运行过程.....	24
3.1 环境准备.....	24

3.2 看它怎么用	24
3.3 实录里的三件事	25
四、本章小结	26
第 5 章 记忆 (Memory) : 让智能体跨会话记住用户	27
一、基本原理	27
1.1 先分清两种记忆	27
1.2 长期记忆就是个文件	27
1.3 记忆的开与关	28
1.4 什么该记, 什么不该记	28
1.5 多用户时要换的东西	28
1.6 依赖环境	28
二、程序介绍	29
2.1 完整程序	29
2.2 开头那五行: 给记忆文件开张	30
2.3 三处改动	30
2.4 章末为什么要把记忆打印出来	30
三、运行过程	30
3.1 环境准备	30
3.2 两个进程, 验一遍	30
3.3 这个测试说明了什么	31
四、本章小结	31
第 6 章 程序执行与沙盒: 让智能体安全地动手干活	33
一、基本原理	33

1.1 三档执行，从安全到危险	33
1.2 为什么不用 Docker	33
1.3 四道闸门	34
1.4 一个坑：两套路径会打架	34
1.5 什么时候用哪一档	34
1.6 几条安全习惯	34
1.7 依赖环境	35
二、程序介绍	35
2.1 完整程序	35
2.2 工具 run_bash：四道闸门里的三道	37
2.3 审批：interrupt_on 与 resume	38
2.4 内存解释器：一行中间件	38
2.5 这一章往程序里加的东西	38
三、运行过程	38
3.1 环境准备	38
3.2 审批：同意执行	39
3.3 审批：拒绝	39
3.4 黑名单拦截	40
3.5 内存解释器	40
3.6 六章下来，minihermes 会什么	40
四、本章小结	40

第 1 章 开发环境搭建与第一个程序

我让 Hermes 帮我查一件事。它会自己搜索，读取网页，在工作目录里写下笔记，最后交给我一份带出处的总结报告。中间还停下来问了一句：这条命令要执行吗？看它跑完，我冒出来的是另一个问题：如果从零开始写这么一个东西，第一行该敲什么。

本章就干这件事，分两部分：先把工作台搭起来（git bash 敲命令、uv 管环境、DeepSeek 作为模型支持），再写出第一个能用的命令行程序——十行代码，先只实现对话，它叫 `minihermes`。学完这一章应该能自己复现一套相同的环境，让程序在终端里连着跟你聊天，并且明白三件事：模型其实什么都不记得、“聊天记录”是怎么来的、为什么说智能体只是“一个循环加一份上下文”。后面五章要加的所有东西，都得从这个十行的内核里长出来。

一、基本原理

1.1 它其实什么都不记得

很多人第一次用大模型，都会以为它“记得”昨天说过的话。

事实是它一点都不记得。模型是无状态的，每次请求都是全新的开始。你可以把它当成一位学识渊博的老先生，答起问题来条理清楚，但他不知道你是谁，也不记得你们五分钟前聊过什么。你想让他接着刚才的话题往下说，就得把前情再讲一遍。

我们也确实是这么干的。所谓“聊天记录”，说白了就是每次请求把之前每一轮的问答重新发一遍。他显得记得，是因为我们在替他记。

从这里能推出三件事，后面几章都会用到：

- 聊得越久，要重发的内容越多，花的 token 越多。所以会有“上下文太长怎么办”这个问题。
- 进程一关，前情就没了，他又变回陌生人。所以会有“记忆”这回事。
- 所谓智能体，并不是更聪明的模型。它只是一个替老先生记事、递工具、跑腿的循环。

1.2 Harness：模型外面那层工程

英文里 harness 是马具、挽具的意思。一副好挽具能把马的气力套住，引到该去的地方。马再壮，挽具不行，车还是跑不起来。

AI 圈把这词借过来，指模型外面那层工程，我觉得挺贴切。模型负责想，想到什么程度是它的本事；外面这层工程负责把想法变成动作。文件怎么读写，工具调错了怎么重试，上下文塞满了怎么压缩，危险操作怎么拦住，都是它的活。

圈里常讲一句话：模型决定上限，Harness 决定下限。日常用下来觉得顺手还是别扭，多半是下限的事。

本书技术栈的主角 DeepAgent (Python 包名 **deepagents**) 就是 LangChain 官方做的一个 Harness。它是个独立的包，底下站在 LangChain 和 LangGraph 上，把下面这些枯燥但又必须的事都办了：

替我们做好的事	具体是什么
文件系统工具	<code>ls</code> 、 <code>read_file</code> 、 <code>write_file</code> 、 <code>edit_file</code> 、 <code>delete</code> 、 <code>glob</code> 、 <code>grep</code>
任务规划	让它先写清单再逐条打勾（需要时挂上对应的中间件）
子智能体	把子任务派给专门的子 agent（ <code>task</code> 工具）
上下文管理	聊天记录太长自动摘要，大结果卸载到文件
技能 Skills	把“这类活怎么干”写成 <code>SKILL.md</code> ，用到时才加载
记忆 Memory	用文件做长期记忆，跨会话不忘
可换后端	文件放内存、放本地磁盘、放沙盒，一句话切换
人工确认	<code>interrupt_on</code> ：指定哪些操作前必须先问你

这副挽具不是一次装成的。往后五章，每一章都往它身上添一样东西。每章尾声我都会留一小段「Harness 视角」，说明这一章添的是哪一层：添之前模型缺什么，添之后挽具管住了什么。

1.3 它有骨架，但没有外壳

有件事得先说清楚，不然到第二章可能会疑惑，那就是：**deepagents** 是个库，不是命令行程序。

经过验证：装完之后 `.venv/Scripts` 里没有任何可执行文件，包也没注册任何命令。
`create_deep_agent(...)` 交给你的是一张编译好的 LangGraph 图，剩下的事情得你自己来。读输入、打印输出、你 > 这个提示符、斜杠命令，全都不在包里。

换句话说，**命令行是我们自己搭的外壳**（本章那十行干的就是这件事），DeepAgent 是往这个外壳里装的骨架。两件事分得很清楚。

（官方另有一个做好的终端智能体叫 Deep Agents Code，命令是 **dcode**，装完敲 **dcode** 就能用，记忆、技能、沙盒、审批都齐了。它把一切都封好，也正因为封得太好，你看不到里面的循环。那是留给我们当对照用的，不是这本书的起点。）

那为什么不干脆用现成的？因为后面五章我们干的事，全都是改这个循环：换工具、塞技能、接记忆、加沙盒。如果连循环长什么样都不知道，改起来就是瞎猜，出了问题也不知道该往哪看。

1.4 全书路线

路线是：**先手写外壳，再往里面装骨架，然后一层层改造它**。每一章都是同一个程序，代码在 `chapterNN/minihermes.py`，可以单独跑，不必先跑前面几章。

章	这一章给 minihermes 加什么本事	这一章添进挽具的那一层
1	开发环境搭建与第一个程序：十行，只会说话 ← 本章	最小的挽具：循环与上下文
2	用 DeepAgent 开发最简智能体：	中间层：循环托管、工具层、文

	会读写文件、会自己拆步骤	件去处
3	工具调用：给智能体接入联网能力	手：自己写的工具，和写给模型的说明书
4	技能（Skills）：把做事的方法固化成能力	手艺与上下文预算：按需加载
5	记忆（Memory）：让智能体跨会话记住用户	状态：会话内与会话外
6	程序执行与沙盒：让智能体安全地动手干活	边界：执行、审批、人在环

书里这个程序叫 `minihermes`。它和 `Hermes` 的关系，大概相当于模型车和真车：架构是通的，尺寸小很多，但每颗螺丝你都亲手拧过。

1.5 依赖环境：三样东西

开工前要准备三样：敲命令的终端、装东西的环境、能说话的模型。

`git bash`。本书所有命令都在 `git bash` 里跑，它是你唯一的终端。

先装 Git for Windows (<<https://git-scm.com/download/win>>，一路默认)。装完不用去开始菜单里找快捷方式：在项目文件夹的空白处点右键，选「Open Git Bash Here」（中文系统里显示「Git Bash Here」），窗口就直接开在这个目录下。验一下：

```
git --version
```

后面凡是说“在终端里运行”、“敲一条命令”，指的都是这个窗口：在项目目录里右键打开它，照着敲。

`uv`。Python 环境的老问题你大概都遇过：装个包把别的项目搞坏了，换台电脑环境重装一下午。`uv` 是用 Rust 写的，一个命令同时管三件事：装 Python、建虚拟环境、装依赖。最大的好处是不碰系统环境，一个项目一个 `.venv` 文件夹，装坏了删掉重来，一分钟。

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version
```

这里交代一个我自己踩的坑。这本书最初我是在 `conda` 环境里起步的，手滑执行了 `pip install --upgrade pip`，`pip` 自带的 `vendored` 依赖当场坏掉，报 `No module named 'pip._vendor.rich.markup'`，修了半天。`uv` 不会出这种事，出了删掉 `.venv` 重来就行。这也是后面全程只用 `uv` 的原因。

DeepSeek 的 Key。 模型我们选 DeepSeek：中文好，便宜，接口兼容 OpenAI。去 <<https://platform.deepseek.com>> 注册登录，在 API Keys 里建一个 Key，形如 `sk-...`。它只显示一次，先复制存好。本章用的模型名是 `deepseek-v4-flash`。

二、程序介绍

2.1 完整程序

新建 `chapter01/minihermes.py`，整份内容如下，一共十行：

```
# minihermes.py — 第 1 章：10 行以内的命令行智能体 运行：uv run python chapter01/minihermes.py
```

```

from dotenv import load_dotenv; load_dotenv()
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="deepseek-v4-flash")          # 读 .env 里的 OPENAI_API_KEY /
OPENAI_BASE_URL
history = []                                           # 会话上下文：一问一答都追加到这里
while (q := input("你 > ")).strip() not in ("", "exit", "quit"):
    history.append({"role": "user", "content": q})      # 记住用户说了什么
    reply = model.invoke(history).content              # 连历史一起发给模型
    history.append({"role": "assistant", "content": reply})
    print("minihermes >", reply, "\n")

```

2.2 逐行讲解

第二行把 `.env` 读进环境变量。第三、四行把模型请进来，Key 和地址都由 `.env` 提供，代码里不出现。

第五行的 `history = []` 是那位老先生的“前情记要”，此刻空的。

第六行开始主循环：打印 `你 >`，等你敲一行。空行、`exit`、`quit` 都算结束对话。`:=` 是海象运算符，边赋值边判断，能省一行。

第七行把你刚说的这句记进前情。第八行把模型叫来，注意递过去的是全部历史，不只是这一句。第九行把它答的话也记下来，下一轮要带上。第十行念给你听。

真正跟模型有关的只有第四行，剩下九行都在做同一件事：替他记着前情，每次重新讲一遍。

2.3 这段程序里没有的东西

十行里没有工具、没有记忆、没有文件读写，也没有任何“智能体”框架的调用。它就是“一个循环加一份上下文”——前面 1.1 节推出来的那个结论，在这里只剩两行 Python。

外壳和骨架的分界线也在这里：这一章我们只搭了外壳。从第 2 章起往壳里装 `create_deep_agent(...)`，壳本身不换。

三、运行过程

3.1 环境准备

挑一个放代码的目录（本书下面统一用 `~/deepagent-course` 表示，你自己换成实际路径），开一个 git bash：

```

cd ~/deepagent-course
uv venv --python 3.12 .venv      # 建虚拟环境；本机没有 3.12，uv 会自己下一个

```

接着在项目根目录写 `pyproject.toml`，把这本书要用的东西声明清楚：

```

[project]
name = "deepagent-course"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "langchain-openai>=1.6.2",      # 和模型说话（DeepSeek 兼容 OpenAI 接口）
    "python-dotenv>=1.2.3",        # 读 .env 里的 Key
    "deepagents[quickjs]>=0.7.15", # 第 2 章的主角；quickjs 是第 6 章的沙盒

```

```

    "playwright>=1.63.0",          # 第 3 章：驱动浏览器搜索
    "trafilatura>=2.2.0",          # 第 3 章：把网页正文抽出来
]

# 国内建议加上，走清华镜像
[[tool.uv.index]]
name = "tuna"
url = "https://pypi.tuna.tsinghua.edu.cn/simple"
default = true

```

清单里后两个包（**playwright**、**trafilatura**）是第 3 章才用到的，这里一并写上。原因很实在：全书的依赖一次装齐，读到后面就不用回头补。你当然也可以只写前三个，到第 3 章再 **uv add playwright trafilatura**，结果一样，只是多敲一次命令。

然后装依赖：

```
uv sync
```

装完会多出一个 **uv.lock**，它把每个包的版本都钉死了。把它一起提交进仓库，将来换电脑或者过半年回来，**uv sync** 一条命令就能还原出今天这个环境。以后加包一律用 **uv add 包名**，不要手改文件。

最后把 Key 放进项目根目录的 **.env**：

```

OPENAI_API_KEY=sk-你的 Key
OPENAI_BASE_URL=https://api.deepseek.com
OPENAI_MODEL=deepseek-v4-flash

```

你可能会奇怪，明明是 DeepSeek，变量名怎么是 **OPENAI_**。因为 DeepSeek 的接口跟 OpenAI 完全兼容，**langchain-openai** 默认就读这几个名字。顺着它的规矩来，代码里就一个字都不用写 Key 和地址，这是后面能压到十行的关键。**.env** 不要提交到仓库，本书的 **.gitignore** 已经去掉了。

3.2 跑起来

```
uv run python chapter01/minihermes.py
```

uv run 会自动找到本项目 **.venv** 里的解释器，不用先 **activate**。这点在 **git bash** 里特别省事：我实测过，**git bash** 作为登录 shell 启动时会重读 profile 把 **PATH** 重置，手动 **activate** 时不时就被冲掉。

下面是我这边的真实对话记录：

```

你 > 你好，用一句话介绍你自己
minihermes > 你好，我是由深度求索公司开发的 AI 助手，可以帮你解答问题、处理信息并陪你聊天。

你 > 我叫小王，在做一套图书管理系统
minihermes > 小王你好！图书管理系统是个很实用的项目，我可以帮你梳理需求、设计数据库、写代码、做界面或排查问题——你现在进行到哪一步了？

你 > 我刚才说我叫什么？
minihermes > 你刚才说你叫**小王**，正在做一套图书管理系统。

你 > exit

```

```
$ uv run python chapter01/minihermes.py
你 > 你好，用一句话介绍你自己
minihermes > 你好，我是一个乐于助人的AI助手，可以帮你解答问题、提供信息并陪你聊天。

你 > 我叫小王，在做一套图书管理系统
minihermes > 小王你好！图书管理系统是个很经典的项目，你目前是想梳理需求、设计数据库，还是需要帮忙写代码或排查问题？告诉你一起推进。

你 > 我刚才说我叫什么？
minihermes > 你刚才说你叫小王。

你 > exit
$
```

上面这张是 `git bash` 窗口里的原样截图，不是手抄的。

三轮对话里有三件事值得留意。

第三轮它答对了名字。它并没有记住你，是我们在第八行把前两轮一起递了过去。想通这个，才算真的理解智能体。

关掉窗口再打开，它会问你“你是谁”。它没变笨，是我们还没给它准备记事本，那是第 5 章的事。

回答里的 ****** 星号是 Markdown 记号，模型习惯这么写。这种小毛病等第 2 章换上正经的 Harness，终端能把格式渲染出来时就顺手修掉了。

四、本章小结

这一章办了三件事。讲清了 DeepAgent 是什么；把工作台支了起来，`git bash` 敲命令，`uv` 管环境，DeepSeek 出脑子，Key 放在 `.env`；写出了第一版 `minihermes`，十行，能连着聊。

它现在的状态是有嘴没手：不会打开文件，不会上网，不会写字，也不会替你跑一段代码。

下一章，命令行还是今天这个命令行，我们往里边装 DeepAgent。装完之后 `minihermes` 会长出文件工具和子智能体，也会第一次在磁盘上留下东西：一个叫 `workspace/` 的目录，里面躺着它自己写的笔记。

Harness 视角：这一章那十行就是一副最小的挽具，它只干两件事——把你说过话按顺序攒起来，攒够了递给模型。整段程序里真正碰到模型的只有 `agent.invoke(messages)` 那一行，另外九行都在替它记着前情。往后每一章，都是往这十行上添零件：添手、添记事本、添规矩。模型的力气没变，挽具越来越像样。

第 2 章 用 DeepAgent 开发最简智能体

第 1 章那个十行的小程序只会对话。这一章我们把 DeepAgent 装进同一个 CLI 里，装完之后仍然什么都不加：不给它自写的工具，不接记忆，不联网。计划很简单，就想看看它自己进来之后什么样。

学完这一章，你手上会有一个会自己列目录、读文件、写文件、遇到多步任务会自己拆着做的迷你智能体；也会知道它进来时带的两个默认设置（没有身份、文件只存在内存里）为什么必须改，怎么各用一行代码改掉。这两件事看着小，后面每一章都建立在它们之上。

一、基本原理

1.1 装进去的是一张图，不是模型

`create_deep_agent(...)` 返回的不是模型，而是一张编译好的 LangGraph 图，里面有循环、有工具节点、有上下文管理。我们要的东西都在里面。第 1 章那句“智能体只是一个循环加一份上下文”，到这一章终于有实物了：循环在包里，上下文管理也在包里，我们写的只是驱动它的那几行。

1.2 外壳不变，变的是中间层

第 1 章我们自己维护 `history`，一轮一轮往里追加。这一章不再手工攒了，交给 `agent` 自己的检查点（`checkpointner`）来管。`config` 里的 `thread_id` 是会话编号：同一个编号就是同一段对话，换一个编号就是另开一局。这也解释了为什么每轮只递一句新输入就够了，前面的它自己记得。

命令行这一层原样保留：还是 `你 >` 提示符，还是那个 `while` 循环。从第 1 章到第 6 章，这一层都不换。

1.3 包里带的七个文件工具

我们一个工具都没写，但它现在有七个，全是包里带的：`ls`、`read_file`、`write_file`、`edit_file`、`delete`、`glob`、`grep`。

其中 `write_file` 和 `edit_file` 的区别值得记一下：前者整份覆盖，后者按片段改。`agent` 改一份长文件时用 `edit_file` 比重新写一遍更省 `token`，也更不容易出错。

有一个常见误会要澄清：任务规划（那个先写 `TODO` 再逐条打勾的本事）不是默认开的，得另外挂一个 `TodoListMiddleware` 才生效。子智能体不一样，它是默认就有的：包里会自动加一个叫 `general-purpose` 的子智能体，所以 `task` 工具开箱就能用（我是在第 4 章那趟运行里看到

它自己派了个子任务才回头查证的，源码里 `graph.py` 那句 `inline_subagents.insert(0, general_purpose_spec)` 就是它）。想换成自己的子智能体，传 `subagents=[...]` 即可。

1.4 它进来时的两个默认

没有身份。 不给 `system_prompt` 时，模型收到的系统提示词是空的。包里确实躺着一份默认提示词（`You are a deep agent, an AI assistant that helps users accomplish tasks using tools...`），但在我们这个情况下没被用上。于是模型手里只有一张工具清单，它会顺着这张清单给自己编一个身份。

文件只在内存里。 默认后端（`StateBackend`）把文件存在 agent 的内存里，会话一结束就没了。想让它落在磁盘上，得换成 `FilesystemBackend`。

这两条都不是 bug，是它把选择权留给了我们。所以本章要拧两颗螺丝：给它身份，给它文件的去处。

二、程序介绍

2.1 完整程序

新建 `chapter02/minihermes.py`，整份内容如下：

```
# minihermes.py — 第 2 章：把 DeepAgent 装进第 1 章那个命令行 CLI 外壳
from pathlib import Path

from dotenv import load_dotenv; load_dotenv()
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

WORKSPACE = Path(__file__).resolve().parent.parent / "workspace"
WORKSPACE.mkdir(exist_ok=True)

model = ChatOpenAI(model="deepseek-v4-flash")
agent = create_deep_agent(
    model=model,
    system_prompt="你是 minihermes，用中文回答，回答简洁。",
    backend=FilesystemBackend(root_dir=str(WORKSPACE)),
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "minihermes"}}

while (q := input("你 > ")).strip() not in ("", "exit", "quit"):
    for step in agent.stream({"messages": [HumanMessage(content=q)]}, config=config,
stream_mode="updates"):
        for update in step.values():
            for m in (update or {}).get("messages") or []:
                for tc in getattr(m, "tool_calls", None) or []:
                    print("  →", tc["name"], str(tc["args"])[0:90])
                if type(m).__name__ == "ToolMessage":
                    print("  ←", str(m.content).replace("\n", " ")[0:90])
                elif getattr(m, "content", ""):
                    print("minihermes >", m.content, "\n")
```

2.2 与第 1 章的差别

大的变化只有两处。

第一，`agent = create_deep_agent(...)` 取代了裸模型。第二，我们不再自己维护 `history`，改由 `checkpointer` 管（见 1.2）。

剩下的都是外壳里的老东西：`load_dotenv()`、`ChatOpenAI`、`while` 循环、你 > 提示符，一行没动。

2.3 那几行 `print` 是干什么的

程序里那段 `print` 是我们自己加的：把每一次工具调用打印出来。不加这段，你只能看到它最后说的话，看不到它中间干了什么。

它靠的是 `stream_mode="updates"`：agent 每走完一步就吐一块更新出来，我们逐块翻一遍，遇到 `tool_calls` 就打印箭头和参数，遇到 `ToolMessage` 就打印工具返回，剩下的是它说的话。这三行循环是本书的“透视眼”，后面几章调试全靠它。

2.4 两颗螺丝

第一颗：身份。

```
system_prompt="你是 minihermes，用中文回答，回答简洁。"
```

你写进去什么，模型就收到什么（这里可以放心：我验过，25 个字符进去，25 个字符出来，没有在背后追加别的东西）。它决定了这个 agent 怎么说话、拿自己当什么。等到第 4 章讲技能时你会看到，有些本事更适合写在技能文件里按需加载，而不是一股脑塞进这里，那是后话。

第二颗：文件的去处。

```
from deepagents.backends import FilesystemBackend
backend = FilesystemBackend(root_dir=str(WORKSPACE))
```

`root_dir` 指向项目根目录下的 `workspace/`。它同时是一道边界：agent 眼里的 `/` 就是这个目录，出不了这个院子。

三、运行过程

3.1 环境准备

本章不需要装新依赖，第 1 章那套环境直接用：

```
cd ~/deepagent-course
uv sync                                # 若是新克隆的仓库，先还原环境
ls .env                               # 确认 .env 还在（里面是 Key 和模型名）
uv run python chapter02/minihermes.py # 能起来就说明环境没问题
workspace/ 目录不用手动建，程序里那两行 WORKSPACE.mkdir(exist_ok=True) 会处理。
```

3.2 第一次跑：把两颗螺丝都松开

先不加 `system_prompt`，也不指定 `backend`，就是最光秃秃的一个 `agent`：

```
agent = create_deep_agent(model=model, checkpointer=InMemorySaver())
```

问它一句：

```
你 > 你好，用一句话介绍你自己
```

```
minihermes > 你好！我是一个 AI 助手，可以帮你查资料、读代码、写文件、跑搜索，处理各种综合性的任务。
```

它说自己是“一个 AI 助手”，能干的事情一口气报了一串：查资料、读代码、写文件、跑搜索。这不是我们告诉它的，是它从工具清单里猜出来的。为确认这件事，我把模型实际收到的系统提示词打出来看了一眼：

```
=== 系统提示词：0 字符 ===
```

空的。 一个字的身份说明都没有。模型手里只有一张工具清单，身份全靠它自己编。

这里还有个更值得记的细节：同一个程序、同一份空提示词，它每次给自己编的名号都不一样。我这边见过“我是一个 AI 助手”，也见过它张口就说“我是 Claude，由 Anthropic 开发的 AI 助手”。注意，喂进去的明明是 DeepSeek。一个模型对自己是谁这件事，就是这么不可靠。

再让它干点活：

```
你 > 把“搭建一个个人博客”拆成三步，写进 plan.md
```

```
→ ls {'path': '/'}
```

```
← No files found
```

```
→ write_file {'file_path': '/plan.md', 'content': '# 搭建个人博客计划\n\n## 第一步：选型与本地搭建\n\n确定技...'}  
← Updated file /plan.md
```

```
minihermes > 已写入 `/plan.md`，三步为：选型与本地搭建 → 内容与样式定制 → 部署与上线。
```

（注：根目录原本是空的，如需放到别的路径告诉我即可。）

它自己调了 `write_file`，写了一份像模像样的计划，还顺带告诉你一句“根目录原本是空的”。（另一次跑的时候它更谨慎，挨个试了 `/tmp`、`/home`、`/workspace` 这些像工作目录的路径，全被挡回来才动手。这种摸索不用你教，是它自己会的。）

但如果你去项目目录里翻，会发现磁盘上什么都没有。文件存在 `agent` 的内存里，会话一结束就没了。

3.3 加上第一颗螺丝：身份

给它一句 `system_prompt`，它立刻换了自我介绍：

```
你 > 你好，用一句话介绍你自己
```

```
minihermes > 你好！我是 minihermes，一个可以读写文件、搜索代码、帮你完成各类文件操作任务的 AI 助手。
```

同一句提问，回答完全不同。这就是系统提示词的分量。

3.4 加上第二颗螺丝：文件落盘

把 `backend=FilesystemBackend(root_dir=str(WORKSPACE))` 也加上，同一个任务再跑一遍。这次 `workspace/plan.md` 真的出现在磁盘上了：

```
$ ls workspace && cat workspace/plan.md
```

```

plan.md
# 搭建个人博客计划

## 第一步：选型与本地搭建
确定技术方案（如 Hugo / Hexo / Next.js），初始化项目，在本地跑通第一个页面。

## 第二步：内容与样式定制
撰写文章、配置分类与标签，挑选并调整主题样式，加上关于页与联系方式。

## 第三步：部署与上线
绑定域名、配置 HTTPS，部署到 GitHub Pages / Vercel 等平台，并接入统计与评论。

$ uv run python chapter02/minihermes.py
你 > 你好，用一句话介绍你自己
minihermes > 你好！我是 minihermes，一个能读写文件、搜索代码、执行多步任务的 AI 助手，有问必答，回答简洁。

你 > 把“搭建一个个人博客”拆成三步，写进 plan.md
→ ls {'path': '/'}
← ['/plan.md']
→ read_file {'file_path': '/plan.md'}
← @@ lines 1-10 of 10 @@ # 搭建一个个人博客 ## 第一步：选型与初始化 确定技术栈（如 Hexo、Hugo 或 Next.js），安装
依
赖并创建本地项目骨
minihermes > plan.md 已存在，里面正好就是这三步：

1. **选型与初始化** — 定技术栈（Hexo / Hugo / Next.js），装依赖、建项目骨架
2. **编写内容与主题** — 配置站点信息，挑主题，写并发布前几篇文章
3. **部署与上线** — 推仓库，用 GitHub Pages / Vercel 自动部署，绑域名开 HTTPS

需要我改写或调整内容吗？

你 > exit
$ ls workspace && wc -c workspace/plan.md
plan.md
458 workspace/plan.md
$

```

这张是 `git bash` 窗口里的原样截图。开头那句自我介绍就是 3.3 节的效果；中间的 `→ ls /→ read_file` 是它调用工具的痕迹（这趟它发现 `plan.md` 已经在，直接读出来核对），`← @@ lines 1-10 of 10 @@` 是工具回话；最后两行是敲 `ls workspace && wc -c workspace/plan.md`，证明文件真的落在磁盘上（458 字节）。

第 1 章结尾我说过它会第一次在磁盘上留下东西，现在兑现了。

3.5 一次多步任务的实录

不写工具照样能干活。给它一个三步任务：

```

你 > 分三步做：先看工作目录里有什么文件，再写一份 notes.md 记录你的观察，最后用一句话总结。
→ ls {'path': '/'}
→ ls {'path': '/workspace'}
minihermes > 我先看看目录。

← Error: Path '/workspace': path_not_found
← ['/plan.md']
→ read_file {'file_path': '/plan.md'}
← @@ lines 1-12 of 12 @@ # 搭建一个个人博客 ### 三步计划 ### 第一步：选型与准备 确定定位与内容
方向...
→ write_file {'file_path': '/notes.md', 'content': '# 观察记录\n\n## 工作目录内容\n- 目录只有
1 个文件: `plan.md` (共 12 行)
← Updated file /notes.md
minihermes > 工作目录只有 `/plan.md`，内容是一个“搭建个人博客”的三步计划（选型准备 → 搭建部署 → 写作
维护），我已把观察写入 `/notes.md`。

```

看那几行 `→` 和 `←`：它先列目录（还顺手试了一下 `/workspace`，撞了个 `path_not_found`），读到 `plan.md` 又打开看了一遍，然后才动笔。那句“我先看看目录”是它自己加的说明，嫌它啰嗦的话，第 4 章会给系统提示词补一句把它管住。

四、本章小结

这一章我们做成的事：把 DeepAgent 装进了第 1 章的外壳，一行 `create_deep_agent(...)` 就够；拧了身份和文件去处两颗螺丝；看着它自己用 `ls`、`read_file`、`write_file` 完成了多步任务。

还有一件事定下来：它现在有七个文件工具，但和我们自己的程序一样，出不了 `workspace/` 这个院子。

下一章给它装上网的能力。搜索按你机器上能跑通的方案来（我这边实测过 DuckDuckGo 连不上，得换一条路），抓到网页正文之后，让它自己去查资料、写报告。

Harness 视角：这一章添的是一整层中间件。第 1 章里“替它记着前情”这件事，现在交给 `checkpoint`，`thread_id` 就是它记账的户头；模型每轮能看到的系统提示词、能调用的七个文件工具、以及能走到哪儿的边界（`workspace/` 这个院子），也全由这一层说了算。第 2 章的 32 行和第 1 章的 10 行比，只多了两处：一行把图装进来，一行把记事本挂上。外壳没动，换掉的是壳里的骨架。

第 3 章 工具调用：给智能体接入联网能力

到第 2 章为止，`minihermes` 是个住在自己院子里的聪明人：它会读写文件，会自己琢磨步骤，但院墙外面的事它一概不知道。你问它一件近来的事，它只有两个选择，说不知道，或者凭记忆编一个。这一章给它装两只手，让它能出去找。

本章手写两个工具：`web_search`（用浏览器搜网页）和 `fetch_url`（读网页正文），都是纯 Python，不要任何搜索服务的 Key，也不碰 Node.js 工具链。学完你应该能自己给智能体添工具，并且明白四件事：工具就是一个普通函数、`docstring` 是写给模型的说明书、工具返回的东西要算进上下文预算、工具失败时要返回能读懂的话而不是抛异常。最后我们会看着它独立查完一轮资料，写出一份带来源的报告。

一、基本原理

1.1 工具就是一个普通函数

先把这个概念说清楚，因为它比看上去简单。

一个工具，就是一个普通的 Python 函数。函数名是工具名，参数是入参，而那个 `docstring` 就是写给模型看的说明书。模型看到的不是代码，是这三样东西：

```
def fetch_url(url: str, max_chars: int = 4000) -> str:
    """读一个网页的正文，返回去掉导航和广告之后的纯文本。
```

```
    什么时候用：已经有具体链接、需要看页面里到底写了什么时。
    长文会被截断，先读前面的部分通常就够判断要不要继续。
    """
```

调用的过程也很朴素，第 2 章你已经见过那两行痕迹了：

```
→ web_search {'query': 'deepagents 是什么 能干什么'}
← 1. Deep Agents overview - Docs by LangChain    https://docs.langchain.com/...
```

→ 是模型发号施令，← 是函数真的跑完、把返回值塞回上下文。一次工具调用就这么完成，没有魔法。

1.2 `docstring` 是产品文案

写工具的时候有个和平时不一样的习惯：`docstring` 不是注释，是产品文案。它得回答模型接下来会问自己的三个问题——这是干什么的、什么时候该用、什么时候别用。写得含糊，模型就会在错误的时机调用它，或者该调的时候不调。

上面那两段工具说明就是按这个思路写的：`fetch_url` 的说明里专门交代了一句“拿到链接之后再用它”，把两个工具的先后顺序讲明白；`web_search` 的说明里则点明“想知道有哪些网页在讲这件事时先用它”。这类句子在普通代码里显得啰嗦，在这里是接口文档。

1.3 搜索为什么自己造

官方文档给的方案是 Tavily（要申请 Key），或者用 Google、OpenAI 自带的搜索工具（要对应厂商的 Key）。两个我都不想要：这本书的前提是一把 DeepSeek Key 走到底。

那就先试最省事的。Python 里有个 `ddgs` 包，抓 DuckDuckGo 的结果，免费不要 Key：

```
ddgs.exceptions.TimeoutException: error sending request for url
(https://html.duckduckgo.com/html/)
超时。在国内连不上。
```

好在第 1 章装依赖时我们已经把 `playwright` 装上了（浏览器还没下，3.1 里补）。既然手上有浏览器，就让它去开必应搜——一样免费，一样不要 Key，还顺便演示了“工具是自己写的”这件事。搜回来的结果再交给 `trafilatura` 抽正文，这两个包都是纯 Python 的。

1.4 返回的东西要进上下文

这里有个地方值得停一下：工具返回的东西是要进上下文的，返回多少就占多少。一次搜索回来十条结果、每条带一整段摘要，几千字就没了；正文整篇塞回去更夸张。所以两个工具都在出口处做了整理——搜索只给 180 字摘要，正文默认截到 4000 字并注明被截断了。

工具写得“话少”，模型后面才有余量干活。

1.5 失败要能读懂

最后，两个工具都得能体面地失败。抓不到页面时我让它返回一句人话，而不是抛异常：

```
if not html:
    return f"抓取失败（页面打不开或超时）：{url}"
```

这一点在三、运行过程里会看到效果：模型收到这句人话，就知道该换一条路；工具要是直接抛异常，任务当时就断了。

1.6 依赖环境

本章不新增依赖：要用到的两个包，第 1 章写 `pyproject.toml` 时就一起声明了，`uv sync` 已经装好。对照一下：

依赖	干什么用	备注
<code>playwright</code>	驱动 Chromium 搜网页	包在环境里，浏览器还要单独下一次： <code>uv run playwright install chromium</code> （见 3.1）
<code>trafilatura</code>	从 HTML 里抽正文	纯 Python，本地解析

只声明了包、没下浏览器的后果很直接：第一次搜索会报找不到浏览器可执行文件。这一步在 3.1 里补。

本机实测的两个前提：DuckDuckGo 在国内连不上（超时），必应可用；Chromium 下到 `%LOCALAPPDATA%\ms-playwright\` 下，之后不再联网。

二、程序介绍

2.1 工具一：web_search

新建 `chapter03/web_tools.py`，第一个函数用浏览器打开必应的结果页，从 DOM 里把标题、链接、摘要抠出来：

```
def web_search(query: str, max_results: int = 5) -> str:
    """用浏览器搜索网页，返回若干条结果的标题、链接和摘要。

    什么时候用：想知道"有哪些网页在讲这件事"、找官方文档、找最新消息时，先用它。
    拿到链接之后，再用 fetch_url 去读其中某一页的正文。
    """
    from playwright.sync_api import sync_playwright

    url = f"https://cn.bing.com/search?q={quote(query)}&setlang=zh-CN"
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page(locale="zh-CN", user_agent=UA)
        page.goto(url, wait_until="domcontentloaded", timeout=30000)
        page.wait_for_selector("li.b_algo", timeout=15000)
        rows = page.eval_on_selector_all("li.b_algo", """els => els.map(e => {...})""")
        browser.close()

    lines = []
    for i, row in enumerate(rows[:max_results], 1):
        if not row.get("url"):
            continue
        lines.append(f"{i}. {row['title']}\n {row['url']}\n {row['snippet'][:180]}")
    return "\n".join(lines) if lines else f"没有搜到「{query}」的结果"
```

两个地方是写浏览器工具最容易栽的坑：`headless=True` 是不要弹出窗口，在后台悄悄搜；`wait_for_selector("li.b_algo")` 是等结果列表真的出现——网页搜索是异步加载的，不等它，抓回来就是一片空白。

抠数据用的是 `eval_on_selector_all`，里面那段 JavaScript 对每条结果取标题、链接和摘要。选 `li.b_algo` 是因为必应把自然结果都放在这个 class 下，比按整页文本猜稳当。文件顶部那串 `UA` 是伪装成普通浏览器，有些站点对无头浏览器不客气。

2.2 工具二：fetch_url

第二个函数短得多，几行就是全部：

```
def fetch_url(url: str, max_chars: int = 4000) -> str:
    """读一个网页的正文，返回去掉导航和广告之后的纯文本。

    什么时候用：已经有具体链接、需要看页面里到底写了什么时。
    长文会被截断，先读前面的部分通常就够判断要不要继续。
    """
    import trafilatura

    html = trafilatura.fetch_url(url)
    if not html:
        return f"抓取失败（页面打不开或超时）：{url}"
    text = trafilatura.extract(html, include_comments=False, include_tables=True) or ""
    text = text.strip()
    if not text:
        return f"这个页面没能抽出正文（可能是纯 JS 渲染的）：{url}"
```

```

if len(text) > max_chars:
    return text[:max_chars] + f"\n\n（正文共 {len(text)} 字，已截断）"
return text

```

三种情况分别对应一句话：抓不到（网络问题）、抽不出（页面是纯 JS 渲染的）、抽出来太长（截断并注明）。每句都能读，模型据此决定下一步。

2.3 主程序

chapter03/minihermes.py 的完整内容：

```

# minihermes.py — 第 3 章：给 minihermes 装上网络（两个工具，不用 Node）
# 运行：uv run python chapter03/minihermes.py
from pathlib import Path

from dotenv import load_dotenv; load_dotenv()
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

from web_tools import fetch_url, web_search # 同目录，直接 import（脚本运行时会自动带上本目录）

WORKSPACE = Path(__file__).resolve().parent.parent / "workspace"
WORKSPACE.mkdir(exist_ok=True)

SYSTEM = (
    "你是 minihermes，用中文回答。"
    "需要查资料时先用 web_search 搜，再挑相关的链接用 fetch_url 读正文，"
    "写进文件的结论要标出来源链接。"
)

model = ChatOpenAI(model="deepseek-v4-flash")
agent = create_deep_agent(
    model=model,
    system_prompt=SYSTEM,
    tools=[web_search, fetch_url],
    backend=FilesystemBackend(root_dir=str(WORKSPACE)),
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "minihermes"}}

while (q := input("你 > ")).strip() not in ("", "exit", "quit"):
    for step in agent.stream({"messages": [HumanMessage(content=q)]}, config=config,
stream_mode="updates"):
        for update in step.values():
            for m in (update or {}).get("messages") or []:
                for tc in getattr(m, "tool_calls", None) or []:
                    print("  →", tc["name"], str(tc["args"])[0:90])
                if type(m).__name__ == "ToolMessage":
                    print("  ←", str(m.content).replace("\n", " ")[0:90])
                elif getattr(m, "content", ""):
                    print("minihermes >", m.content, "\n")

```

2.4 与第 2 章的差别

实质变化只有一行，把两个工具交上去：

```
tools=[web_search, fetch_url], # ← 新的
```

系统提示词里加了一句交底：

```
SYSTEM = (
```

```

"你是 minihermes，用中文回答。"
"需要查资料时先用 web_search 搜，再挑相关的链接用 fetch_url 读正文，"
"写进文件的结论要标出来源链接。"
)

```

最后半句是给自己留的退路。模型默认不太爱标来源，而在这种“查了五个网页写一份报告”的活里，没有来源的报告等于没法核对。

三、运行过程

3.1 环境准备

```

cd ~/deepagent-course
uv sync                                # 还原环境（会装上 playwright 和 trafilatura）
uv run playwright install chromium     # 下浏览器，只在第一次需要（约 150MB）
uv run python chapter03/minihermes.py # 能起来就说明依赖完整了

```

第二条命令如果跳过，第一次搜索会报找不到浏览器可执行文件。装好的浏览器

在 `%LOCALAPPDATA%\ms-playwright\` 下，之后离线也能跑。

3.2 一次真实调研

给它一个真活儿：

你 > 查一下 deepagents 是什么、能干什么，写成 report.md，来源链接放最后。

下面是这一趟的原样记录——模型每次说话用词都会有出入，你照着跑不会和我这里一字不差，动作顺序是一致的：

```

你 > 查一下 deepagents 是什么、能干什么，写成 report.md，来源链接放最后。
→ web_search {'query': 'deepagents', 'max_results': 6}
→ web_search {'query': 'langchain deepagents 是什么', 'max_results': 6}
minihermes > 我先搜一下 deepagents 的相关资料。
  ← 1. LangChain 中文教程 | LangChain 中文文档 https://langchain-doc.cn/
  ← 1. Deep Agents overview - Docs by LangChain
https://docs.langchain.com/oss/python/deepa...
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/overview',
'max_chars': 8000}
→ fetch_url {'url': 'https://github.com/langchain-ai/deepagents', 'max_chars': 8000}
minihermes > 官方文档和 GitHub 仓库看起来是最权威的，我读一下正文。
  ← - Take actions in an environment: Take actions via tools, read and write files, execute
CO...
  ← 抓取失败（页面打不开或超时）：https://github.com/langchain-ai/deepagents
→ fetch_url {'url': 'https://raw.githubusercontent.com/langchain-
ai/deepagents/main/README.md', ...
→ fetch_url {'url': 'https://docs.langchain.org.cn/oss/python/deepagents/overview', ...
minihermes > GitHub 打不开，我换 raw README，并补读文档后半部分（子代理、人在环等）。
  ← 这个页面没能抽出正文（可能是纯 JS 渲染的）：https://docs.langchain.org.cn/oss/python/...
  ← 抓取失败（页面打不开或超时）：https://raw.githubusercontent.com/...
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/overview',
'max_chars': 24000}
→ fetch_url {'url': 'https://www.runoob.com/ai-agent/deep-agents-usage.html', 'max_chars':
6000}
→ fetch_url {'url': 'https://pypi.org/project/deepagents/', 'max_chars': 3000}
minihermes > 再确认一下 PyPI 上的包信息（版本、安装方式）。
→ write_file {'file_path': '/report.md', 'content': '# Deep Agents (deepagents) 调研报告
\n\n## 一、它是什么...
  ← Updated file /report.md

```

它最后交出来的是 `workspace/report.md`，一份有分节、有来源链接的调研报告。

```

→ fetch_url {'url': 'https://www.runoob.com/ai-agent/deep-agents-usage.html'}
→ fetch_url {'url': 'https://pypi.org/project/deepagents/'}
← Deep Agents 入门教程 Deep Agents 是 LangChain 官方推出的一套开箱即用 (Batteries Included) 的 Agent 开发框架，专
面向复
← ??? Deep Agents Looking for the JS/TS version? Check out Deep Agents.js. To help you ship L
→ ls {'path': '/'}
← ['/plan.md']
→ read_file {'file_path': '/plan.md'}
← @@ lines 1-10 of 10 @@ # 搭建一个个人博客 ## 第一步：选型与初始化 确定技术栈（如 Hexo、Hugo 或 Next.js），安装
依赖并创建本地项目骨
→ write_file {'file_path': '/report.md', 'content': '# Deep Agents 调研报告\n\n## 一、它是什么\n\n*Deep Agents 是
← Updated file /report.md
minihermes > 已写好 /report.md。核心结论：

**是什么**：Deep Agents 是 LangChain 官方开源的 **agent harness（智能体运行框架）**，PyPI 包名 `deepagents`，官方定位
是“opinionated、开箱即用的 Agent，且任何一部分都可扩展/替换”。它受 Claude Code 启发，用的是和目标框架一样的 tool calli
ng 循环，但把长期任务的脚手架内置了。

**能干什么**：
- **执行环境**：工具（含任意 MCP server）、虚拟文件系统、沙箱、代码执行/REPL，带路径级权限控制
- **上下文管理**：Skills 按需加载、Memory 记忆、自动摘要、大结果卸载到磁盘、prompt 缓存
- **委派**：子 Agent（独立上下文窗口）派生 + 用 `write_todos` 做任务规划
- **引导**：human-in-the-loop 审批与中断
- 附带 5 种可插拔文件系统后端（State / Filesystem / Store / LocalShell / Composite）

**与 LangChain、LangGraph 的关系**：三者是同一栈的不同层——LangChain 提供积木，LangGraph 是图运行时，Deep Agents 是
`create_agent` 之上更有主见的 harness；任何 LangGraph `CompiledStateGraph` 都能作为 subagent 塞进去。

**注意点**：`FilesystemBackend` / `LocalShellBackend` 会直接操作本地磁盘和 shell，需先配权限；HITL 要配 checkpointing；`d
elete` 工具与 `tools` 白名单需 `deepagents>=0.7`，且白名单必须保留 `read_file`。

来源（放在文末）：LangChain 官方文档 overview、GitHub 仓库 README、PyPI 项目页、菜鸟教程中文入门教程。GitHub 页面直接
抓取超时，README 内容是通过 raw 链接拿到的。

你 > exit
$

```

git bash 窗口里的原样截图。这一趟的窗口放不下全部输出，图上能看到的是后半程：几轮 `fetch_url` 之后写文件、以及它最后的总结。

3.3 实录里的三件事

它一次并发发了两个搜索。模型在一轮里可以吐出多个工具调用，框架会一起跑。搜索这种互不依赖的活，并发起来省时间。

它失败了四次，自己换了三条路。GitHub 页面抓不到（纯 JS 渲染），它换 raw README；raw 也失败，它转去读中文文档站；又失败，最后靠官方文档 + 菜鸟教程 + PyPI 页面凑齐。每一次失败工具返回的都是“抓取失败（页面打不开或超时）”这种能读懂的话，所以模型知道该换路。这就是 1.5 节那条规矩的回报。

它自己给报告加了“提醒”。收尾那段话里写着“GitHub 和 raw README 当时抓取失败，仓库信息取自搜索结果摘要；官方示例中的模型名是抓取原文，实际用你自己的模型替换”。我没有让它写这些，是它自己判断该交代。

3.4 现阶段的不足

一是它话多。每做几个动作就要来一句“我先搜一下”“再确认一下”，这些解说既占屏幕也占上下文。这一章先不管它，第 4 章给系统提示词加一句“干活过程中不要输出这类说明”，输出立刻就干净了。

二是抓取失败率高不高，取决于对方网站的脾气。纯 JS 渲染的站点 `trafilatura` 抽不出东西，这是它天生的短板——实测里 GitHub、知乎、中文镜像站三个都栽在这上面。真需要读这类

页面，可以在 `fetch_url` 里换成“用 Playwright 打开、等渲染完再取文本”，代价是慢好几倍。这一章先不折腾。

四、本章小结

这一章 `minihermes` 有了两只手：`web_search` 搜，`fetch_url` 读。两个工具都是自己写的，纯 Python，没有引入任何 Key 和 Node 工具链。

顺带立下了写工具的几条规矩，后面几章都照这个来：`docstring` 当产品文案写、返回结果要控量、失败要返回人话。

现在的它：会在 `workspace/` 里读写文件，会自己上网查资料，写出来的东西会标来源。

下一章处理一个更容易被忽视的问题：“查资料”这件事，它每次都要自己重新摸索一遍流程。搜几个词、看哪些链接、报告写成什么结构——这些都是可以固化成“技能”的。我们会给它一份 `SKILL.md`，让它在需要的时候自己翻出来照着做，顺便聊聊技能怎么“自己长出来”。

Harness 视角：这一章添的是手。模型自己既不会搜索也不会开网页，它能做的只是决定“我要调 `web_search`，参数是这么几个词”；把请求发出去、把网页抓回来、把噪音滤掉，全是挽具这一侧的 Python 函数在干。还有个容易忽略的细节：模型只能看见 `docstring` 里写的那几行，所以写工具这件事，一半是写代码，另一半是替模型写说明书。返回留多长、失败怎么说，同样是挽具作者定的。

第 4 章 技能 (Skills)：把做事的方法固化成能力

第 3 章的 `minihermes` 查资料已经很能干了，但有个问题：它**每次都在重新摸索**。搜几个词、挑哪几个链接、报告写成什么结构，全看它当时的心情，同一个任务跑两遍，两遍的结构都不一样。这不是它笨，是我们没把“这类活该怎么干”教给它。

这一章就来教。我们写一份 `SKILL.md`（一份写给模型看的操作手册），把它挂到智能体的技能目录上，让它在遇到这类任务时自己去翻、照着做。学完你应该能自己写技能，并且明白四件事：技能和工具的分工、`description` 为什么是唯一的广告位、三层渐进加载怎么省上下文、以及“让技能自己长出来”这件事为什么必须先解决控制权。

一、基本原理

1.1 工具是手，技能是手艺

第 3 章装的是工具：`web_search` 会搜，`fetch_url` 会读。工具管的是“能做什么”。但“查一份资料并写成报告”这件事，光有手不够，还得知道套路：先定几组关键词、来源按什么优先级挑、结论要几处来源互证、报告要有哪几节、拿不准的怎么写。这些是手艺。

在 DeepAgent 里，手艺叫**技能 (Skill)**。它和工具的分工很清楚：

	是什么	什么时候进上下文
工具	一个能调用的函数	一开始就在（模型得知道手里有什么）
技能	一份“这类活怎么干”的说明书	需要时才读

这个区别很关键。工具得一直挂着，不然模型不知道该调谁；技能不用——它太多了，全塞进去会把上下文撑爆。

1.2 一个技能长什么样

就是一个目录，里面放一份 `SKILL.md`：

```
skills/
├── web-research/
│   ├── SKILL.md           ← 必须
│   └── references/
│       └── report-template.md ← 可选，用到时才会被读
```

`SKILL.md` 开头的 YAML 头 (frontmatter) 只有两个必填字段：`name` 和 `description`。
`description` 是这份技能唯一的“广告位”。模型启动时只能看到每个技能的 `name` 和 `description`，它靠这一句话判断“这次要不要翻这份说明书”。写得含糊，技能就等于没装。

1.3 三级渐进加载

这是技能设计的精髓，也是它跟“把一大段提示词塞进系统提示词”的区别。技能分三层往里进：

层	读什么	什么时候
一	SKILL.md 的 name + description	agent 启动，全部技能都读
二	SKILL.md 正文	这个技能被“想起来”的时候
三	references/ 、 scripts/ 、 assets/ 里的附件	正文里点名要它读的时候

所以你可以放心往里加技能：启动时它们只占两行字，真正的说明书躺在磁盘上。等到三、运行过程你会看到，它先读了 SKILL.md，然后才去读 **references/report-template.md**，三层的顺序清清楚楚。

1.4 技能是作者写的东西，只读

技能会被当指令执行（模型会照着做），所以它是“手艺”，不是“草稿”。本章的配置里有一条权限，把 **/skills/**** 的写操作全部拒掉：agent 只许读、不许改。

这条权限还有一个更重要的用途，见下一节。

1.5 自进化与三道闸门

前面说技能是作者写的东西。那能不能让 agent 自己写？

想法很自然：它刚跑完一次成功的调研，流程里有些经验（比如“这个站纯 JS 渲染，抓不到正文，别浪费时间”），让它把这些写进技能目录，下次就少走弯路。

这个方向叫自进化，听起来很美，但真正危险的地方在于“它写的东西下次会直接生效”。技能是会被当指令执行的，一旦写歪了，或者被人诱导写了坏内容，下一轮就带病运行。这不是假想，第 3 章的抓取结果里就混进来过一条抖音视频的标题——那是搜索结果的噪声，要是它把这种内容当经验写进技能，麻烦就大了。

所以我建议这么设计，三道闸门，代码上都不难：

第一，技能目录保持只读（本章的 **permissions** 就是这么配的）。想让它写，另开一个草稿区，比如 **/drafts/**，和正式的 **/skills/** 物理隔开。

第二，写草稿要人点头。 第 6 章我们会给 **run_bash** 加审批，同一套机制放在这里：
interrupt_on={"write_file": True}，它想写草稿时先弹给你看。

第三，草稿转正必须过人的眼睛。 这一步我推荐最土也最靠得住的办法：**git diff**。技能目录在 git 里，agent 写的草稿提交成一次 commit，你打开一看就知道它想教你什么。看得顺眼就合进 **skills/**，看不顺眼就删。

说到底，自进化的门槛不在技术，在**控制权**：写到哪里、什么时候生效、谁说了算。这三件事想清楚了，自进化才是个好东西。

1.6 依赖环境

本章不新增任何依赖，第 3 章的环境继续用。新增的是文件，不是包：

```
chapter04/
├── minihermes.py          ← 主程序
├── web_tools.py           ← 第 3 章那两个工具，原样拷过来
├── skills/web-research/
│   ├── SKILL.md
│   └── references/report-template.md
```

技能目录放在章节里（不是工作目录），靠下面的 **CompositeBackend** 挂进去。

二、程序介绍

2.1 技能文件：SKILL.md

chapter04/skills/web-research/SKILL.md 的完整内容：

```
---
name: web-research
description: 需要上网查资料并写成一份带来源的报告时使用。给出关键词策略、来源筛选、报告结构和不确定项的写法。
---

# 网络调研（web-research）

## 什么时候用

用户让你"查一下某件事，然后写成报告 / 总结 / 对比材料"时。

如果只是随口问一句、不需要落成文件，直接回答就行，不必走这套流程。

## 步骤

1. **定关键词**。想 2~3 组词，一组问"是什么"，一组问"怎么用"，必要时再加一组"有什么坑"。只搜一个词通常搜不到有用的东西。
2. **搜索**。用 `web_search` 把这几组词都搜一遍，可以在一轮里同时发多个搜索。
3. **挑来源**。从结果里挑 2~3 个，优先级：官方文档 > 官方仓库 > 权威社区 > 个人博客。聚合站、SEO 农场一律跳过。
4. **读正文**。对挑中的链接用 `fetch_url` 读正文。读 2~3 篇就够，不要把搜索结果挨个读完。
5. **交叉核对**。关键结论至少要在两处来源都出现过才写进报告；只有一处来源的，写的时候标明"仅见于某来源"。
6. **写报告**。按 `references/report-template.md` 里的结构写，存成 `.md` 文件。
7. **收尾**。报告末尾必须有"来源链接"一节；有拿不准的地方，单开"备注与不确定项"一节写清楚。

## 别做的事

- 不要把搜索结果的摘要原样贴进报告，要读过正文再写。
- 不要编来源。没读到正文的链接，不能当依据写结论。
- 不要写"根据某某网站"这种模糊说法，要给具体链接。
```

正文就是写给模型看的操作手册，我这份写得很直白。注意第 6 步——它点名让模型去读第三层的附件。

2.2 附件: references/report-template.md

技能目录里的 `references/` 放的是“用到才读”的附件。这个报告模板定义了五节结构加一节来源:

```
# <题目>调研报告

## 一句话结论
## 它是什么
## 怎么用 / 关键细节
## 适用场景与注意事项
## 备注与不确定项
## 来源链接
```

文件末尾还写了三条硬要求, 其中一条是“抓取失败的来源也要列出来, 并在不确定项里说明, 别悄悄删掉”。这条在第 3 章里是被模型自己悟出来的, 现在写进附件, 就成了固定流程。

2.3 主程序

`chapter04/minihermes.py` 的完整内容:

```
# minihermes.py — 第 4 章: 给它一份技能 (SKILL.md), 让它按套路干活
# 运行: uv run python chapter04/minihermes.py
from pathlib import Path

from dotenv import load_dotenv; load_dotenv()
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, FilesystemBackend

from web_tools import fetch_url, web_search

HERE = Path(__file__).resolve().parent          # chapter04/
WORKSPACE = HERE.parent / "workspace"
WORKSPACE.mkdir(exist_ok=True)

SYSTEM = (
    "你是 minihermes, 用中文回答。工具和技能的说明都在你的上下文里, 照着用。"
    "干活过程中不要输出「让我先看看.....」这类说明, 直接做, 最后再给结论。"
)

model = ChatOpenAI(model="deepseek-v4-flash")
agent = create_deep_agent(
    model=model,
    system_prompt=SYSTEM,
    tools=[web_search, fetch_url],
    backend=CompositeBackend(
        default=FilesystemBackend(root_dir=str(WORKSPACE)),          # 干活的地方
        routes={"/skills/": FilesystemBackend(root_dir=str(HERE / "skills"))}, # 技能目录挂到
    ),
    skills=["/skills/"],          # 技能只在需要时才加载 (启动只读每个技能的 name 和
description)
    permissions=[
        FilesystemPermission(operations=["write"], paths=["/skills/**"], mode="deny"),
    ],          # 技能是“手艺”, 只读, 不许 agent 自己改
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "minihermes"}}
```

```

while (q := input("你 > ")).strip() not in ("", "exit", "quit"):
    for step in agent.stream({"messages": [HumanMessage(content=q)]}, config=config,
stream_mode="updates"):
        for update in step.values():
            for m in (update or {}).get("messages") or []:
                for tc in getattr(m, "tool_calls", None) or []:
                    print("  →", tc["name"], str(tc["args"][:90]))
                if type(m).__name__ == "ToolMessage":
                    print("  ←", str(m.content).replace("\n", " ")[:90])
                elif getattr(m, "content", ""):
                    print("minihermes >", m.content, "\n")

```

2.4 三处改动各解决什么问题

相比第 3 章，新增的就是 **backend** 里的路由、**skills** 和 **permissions** 这三处。

CompositeBackend 是干吗的？第 2 章我们用 **FilesystemBackend** 把 agent 的根目录指到 **workspace/**。技能文件在项目里（**chapter04/skills/**），不在工作目录里，所以需要有一个“路由”：默认路径走工作目录，**/skills/** 开头的走技能目录。这样技能目录不用搬进工作区，也不会被 agent 顺手改掉。

skills=["/skills/"] 是唯一的开关，路径用正斜杠、相对后端的根。

permissions 那一行把 **/skills/**** 的写操作全部拒掉，理由见 1.4 和 1.5。

顺带说一句系统提示词：这一章加了一句“干活过程中不要输出「让我先看看……」这类说明”。第 3 章那个“话多”的毛病，一句话就治好了。

三、运行过程

3.1 环境准备

```

cd ~/deepagent-course
uv sync                                # 无新依赖，这条是保险
ls chapter04/skills/web-research/SKILL.md  # 确认技能文件在
uv run python chapter04/minihermes.py

```

启动后它会读一遍 **/skills/** 下每个技能的 **name** 和 **description** 配进上下文，你看不到这个过程，但等下问对问题，它就会去找。

3.2 看它怎么用

给它一个任务，看它动不动那份说明书：

```

你 > 查一下 deepagents 的 Skills 是什么、怎么用，写成 report.md
→ read_file {'file_path': '/skills/web-research/SKILL.md', 'limit': 1000}
→ web_search {'query': 'deepagents Skills', 'max_results': 8}
← @@ lines 1-28 of 28 @@ --- name: web-research description: 需要上网查资料并写成一份带来源的报告时使用...
← 1. Deep Agents overview - Docs by LangChain
https://docs.langchain.com/oss/python/deepa...
→ read_file {'file_path': '/skills/web-research/references/report-template.md', 'limit': 1000}
→ web_search {'max_results': 8, 'query': 'deepagents skills SKILL.md progressive disclosure'}

```

```

→ web_search {'max_results': 8, 'query': 'deepagents 怎么用 Skills 教程'}
← @@ lines 1-40 of 40 @@ # 报告结构模板 写报告时照这个骨架填...
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/skills', 'max_chars': 8000}
→ fetch_url {'url': 'https://github.com/langchain-ai/deepagents', 'max_chars': 6000}
← 抓取失败（页面打不开或超时）: https://github.com/langchain-ai/deepagents
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/skills.md', 'max_chars': 30000}
→ fetch_url {'url': 'https://agentskills.io/specification', 'max_chars': 8000}
← Directory structure A skill is a directory containing, at minimum, a SKILL.md file...
→ ls {'path': '/'}
← ['/skills/']
→ write_file {'file_path': '/report.md', 'content': '# Deep Agents Skills 调研报告\n\n 调研时间: 基于 2026 年...'}
← Updated file /report.md
← @@ lines 1-40 of 40 @@ # 报告结构模板 写报告时照这个骨架填。每一节的标题可以直接用，也可以按题目改得更贴合内容

'''markdown # <题目>
→ web_search {'query': 'deepagents skills 是什么', 'max_results': 5}
→ web_search {'query': 'langchain deepagents skills how to use', 'max_results': 5}
→ web_search {'query': 'deepagents skills SKILL.md progressive disclosure', 'max_results': 5}
← 1. https://docs.langchain.com/oss/python/deepagents/overview 2. https://zhuanlan
← 1. Deep Agents overview - Docs by LangChain https://docs.langchain.com/oss/python/deepa
← 1. LangChain 中文教程 | LangChain 中文文档 https://langchain-doc.cn/ LangChain 是智能体工程 (agent
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/overview', 'max_chars': 6000}
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/skills', 'max_chars': 8000}
→ web_search {'query': 'deepagents skills 常见问题 skills not loaded 排查', 'max_results': 6}
← 1. Deep Agents overview - Docs by LangChain https://docs.langchain.com/oss/python/deepa
← - Take actions in an environment: Take actions via tools, read and write files, execute co
← Usage 1 Create a top-level skills directory Create a directory to hold all skills for your
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/skills', 'max_chars': 20000}
→ fetch_url {'url': 'https://docs.langchain.org.cn/oss/python/deepagents/skills', 'max_chars': 10000}
→ web_search {'query': 'deepagents skills 常见问题 skills not loaded 排查', 'max_results': 6}
← Usage 1 Create a top-level skills directory Create a directory to hold all skills for your
← 1. Deep Agents overview - Docs by LangChain https://docs.langchain.com/oss/python/deepa
← 这个页面没能抽出正文（可能是纯 JS 渲染的）: https://docs.langchain.org.cn/oss/python/deepagents/skills
→ fetch_url {'url': 'https://datawhalechina.github.io/deepagents-in-action/', 'max_chars': 5000}
→ fetch_url {'url': 'https://www.runoob.com/ai-agent/deep-agents-usage.html', 'max_chars': 6000}
→ web_search {'query': 'deepagents create_deep_agent skills=["/skills/"] example FilesystemBackend', 'm
← Deep Agents 入门教程 Deep Agents 是 LangChain 官方推出的一套开箱即用（Batteries Included）的 Agent 开发框架，专
面向复
← 1. Deep Agents overview - Docs by LangChain https://docs.langchain.com/oss/python/deepa
← 抓取失败（页面打不开或超时）: https://datawhalechina.github.io/deepagents-in-action/
→ fetch_url {'url': 'https://docs.langchain.com/oss/python/deepagents/skills.md', 'max_chars': 20000}
→ fetch_url {'url': 'https://github.com/langchain-ai/deepagents', 'max_chars': 8000}
← > ## Documentation Index > Fetch the complete documentation index at: https://docs.langcha
← 抓取失败（页面打不开或超时）: https://github.com/langchain-ai/deepagents
→ fetch_url {'url': 'https://agentskills.io/specification', 'max_chars': 8000}
→ web_search {'query': 'langchain-ai langchain-skills repository 官方技能库', 'max_results': 5}
→ task {'description': '请用 fetch_url 抓取 https://docs.langchain.com/oss/python/deepagents/skills.m
← Directory structure A skill is a directory containing, at minimum, aSKILL.md file: SKILL.m
← 1. LangChain 中文教程 | LangChain 中文文档 https://langchain-doc.cn/ LangChain 是智能体工程 (agent

```

git bash 窗口里的原样截图。这一趟它查得很久，图上是中段：最上面那行 ← @@ lines 1-40 of 40 @@ # 报告结构模板 是它翻到技能第三层附件的结果（技能正文里写着“按 references/report-template.md 里的结构写”），中间一大片 → web_search /→ fetch_url 是它在反复换路子捞资料，倒数第二行那个 → task {'description': ...} 是它把一部分抓取活派给了子智能体。

它第一步读 SKILL.md 那下没截进去（滚出窗口了），把本节的任务自己跑一遍就能看到，上面那段记录里也记着。

3.3 实录里的三件事

看上面那段记录的第一行：任务刚进来，它第一个动作就是去读 SKILL.md（换一趟运行，措辞会有出入，动作顺序一样）。这是 description 起作用了——“需要上网查资料并写成一份带来源的报告时使用”，跟这次的任务对上了。

然后是第三层的证据：读完 SKILL.md，它接着读了 references/report-template.md。因为正文第 6 步写着“按 references/report-template.md 里的结构写”。三层加载，一层不落。

中间还有两处值得看。一处是它跑到 agentskills.io (Agent Skills 规范的官方站) 去核对 frontmatter 的写法——技能里只说“按模板写报告”，没让它去核规范，这是它自己加的严谨。另一处是收尾时它列了个“不确定项”，写着“GitHub、知乎、中文镜像三个来源抓取失败，已在报告中如实列出而非隐去”。这句话是我没想到的，也正是技能里那句“别编来源”想要的效果。

这就是技能的价值：同一件事，昨天它是一个灵光一闪的搜索，今天它是一套会自己走完的流程。写一次，以后每次都这么干。

四、本章小结

这一章给 minihermes 装上了手艺：它会认技能，会在需要的时候自己去读说明书，会照着步骤干活。一起立下的还有三条规矩：**description** 当广告位写、附件按需加载、技能目录只读。

现在的它：能读写文件、能上网查资料、能按套路写报告。它还差两样东西——一是记不住你（重启就忘），二是干不了活（只会写文件，不会跑命令）。

下一章先解决第一个：记忆。你会看到它的记忆就是磁盘上一个普普通通的 markdown 文件，以及“新起一个进程还能不能认出你”这个测试。

Harness 视角：这一章添的是手艺，还有一套上下文预算的做法。工具和技能分工不同：工具管“能不能做”，技能管“按什么套路做”。三级加载是这一层最值得记的取舍——开机时只把 **description** 那一行塞进上下文，真要干活了才让模型去读整份 **SKILL.md**。上下文是挽具手里最紧的资源，谁往里塞、塞多少，是挽具作者要算的账，模型算不了。技能目录设成只读也是一个道理：能读不能改，模型就动不了自己的说明书。

第 5 章 记忆 (Memory)：让智能体跨会话记住用户

第 1 章那个十行程序，关掉窗口就把你忘了。此后的 `minihermes` 一直没改这个毛病：每启动一次，它都是个陌生人，你得重新介绍自己、重新交代偏好。这一章治这个病。

做法很朴素：让它在磁盘上有一个记事本，开机先读一遍，聊天中遇到值得长期记住的事就写进去。学完你应该能做到四件事：分清“会话内记忆”和“跨会话记忆”、亲手配一个跨进程生效的记忆、知道记忆里该记什么不该记什么、以及多用户场景下该把它换成什么后端。最后我们用最笨也最靠得住的办法验收：关掉程序，新起一个进程，问它你是谁。

一、基本原理

1.1 先分清两种记忆

“记忆”这个词在智能体里指两件不同的事，混在一起就会乱。

一种是会话内的记忆，也就是一段对话里它记得你上一句说了什么。这个第 2 章就解决了：靠 `checkpointer` 加上 `thread_id`，同一个编号就是同一段对话，进程在跑，它就一直记得。

另一种是跨会话的记忆，进程重启、换台机器、明天早上再来，它还认识你。这是我们这一章要的。

打个比方：前一种像当前这通电话里聊过什么，后一种像通讯录。挂了电话，通话内容该忘就忘，但通讯录得留着。

1.2 长期记忆就是个文件

很多框架把长期记忆做成数据库，要配服务、要建表。`DeepAgent` 的做法朴素得多：记忆是文件。

我们这一章的记忆，就是磁盘上这一个文件：`workspace/memories/AGENTS.md`。

选文件而不是数据库，我图的是三件事：你随时能打开看、能用 `git` 管起来、出问题能手动改。长期记忆这种东西，看得见比什么都重要——不然哪天它记错了什么，你连去哪儿改都不知道。

`AGENTS.md` 这个名字不是我们发明的，它是这套生态里的约定（第 2 章装 `deepagents` 时就提到过），别的工作流也认这个文件。

1.3 记忆的开与关

开关是一个参数：`memory=["/memories/AGENTS.md"]`。agent 启动时会把这个文件的内容读进来，当作既定事实带上。所以它“开口就知道你是谁”，不是记性好，是开机就把通讯录看了一遍。

另一半在系统提示词里。不写的话，模型未必会主动去翻这个文件、也未必想到该往里写。这跟第 2 章给它身份是一个道理：你不说，它就自己猜。

1.4 什么该记，什么不该记

记忆不是越多越好。文件写满了，占的是每次对话的上下文，还会把有用的信息淹掉。我自己的规矩是三条：

该记的：稳定的偏好（“回答用中文、尽量短”）、身份与背景（你是谁、在做什么项目）、长期约定（“报告里必须标来源”）。

别记的：一次性的事实（“今天下午三点开会”）、能现查的（天气、股价）、大段原文（那是文件该干的活，不是记忆）。

冲突了就改，不要追加。“我换工作了”应该把旧的那条改掉。你要是只在系统提示词里说一句“记住用户的话”，它很快会把同一个事实记成三个互相矛盾的版本——这也是为什么下面我们特意写的是“用 `edit_file` 写进去”，而不是“往文件里加一行”。

1.5 多用户时要换的东西

我们这一章的记忆是**全机共用的一份文件**，一个人用、一台电脑上跑，够用。真要做成多用户的東西，DeepAgent 提供的是另一套：把记忆放进 LangGraph 的 store，用命名空间分开。

```
StoreBackend(namespace=lambda rt: (rt.server_info.user.identity,))    # 每个用户一份
StoreBackend(namespace=lambda rt: (rt.server_info.assistant_id,))    # 每个 agent 一份
```

用户 A 的偏好不会流到用户 B 那里，几个 agent 也能各记各的。这种场景下还要多加一道防线：**记忆是会被当指令读的，谁都能写的地方就有注入的风险**。所以共享的记忆通常配成只读（用第 4 章那条 `FilesystemPermission` 就能做到），要写就得走审批。

至于怎么把它接起来——`CompositeBackend` 加路由，把 `/memories/` 指到 store 后端，做法和上一章挂技能目录一模一样。留给你自己动手，比抄一遍有用。

1.6 依赖环境

本章不新增依赖，也不用改 `pyproject.toml`。新增的是一个文件和一处配置：

- 文件：`workspace/memories/AGENTS.md`（程序启动时自动创建，不用手动建）
- 配置：`create_deep_agent(..., memory=["/memories/AGENTS.md"])`

路径写的是 `/memories/...`，这是 agent 眼里的路径——它的根就是 `workspace/`（第 2 章定的规矩）。

二、程序介绍

2.1 完整程序

chapter05/minihermes.py 的完整内容:

```
# minihermes.py — 第 5 章: 让它记住你 (跨进程的长期记忆)
# 运行: uv run python chapter05/minihermes.py
from pathlib import Path

from dotenv import load_dotenv; load_dotenv()
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

from web_tools import fetch_url, web_search

HERE = Path(__file__).resolve().parent
WORKSPACE = HERE.parent / "workspace"
MEM_FILE = WORKSPACE / "memories" / "AGENTS.md"      # 长期记忆就住在这个文件里
MEM_FILE.parent.mkdir(parents=True, exist_ok=True)
if not MEM_FILE.exists():
    MEM_FILE.write_text("# 关于用户的长期事实\n", encoding="utf-8")

SYSTEM = (
    "你是 minihermes, 用中文回答, 回答尽量简短。"
    "/memories/AGENTS.md 里记的关于用户的长期事实, 每次回答前先看一眼;"
    "用户说了值得长期记住的信息, 就用 edit_file 写进去。"
)

agent = create_deep_agent(
    model=ChatOpenAI(model="deepseek-v4-flash"),
    system_prompt=SYSTEM,
    tools=[web_search, fetch_url],
    backend=FilesystemBackend(root_dir=str(WORKSPACE)),
    memory=["/memories/AGENTS.md"],      # 启动时读进来, 干活时可以改
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "minihermes"}}

def turn(text: str) -> None:
    for step in agent.stream({"messages": [HumanMessage(content=text)]}, config=config,
stream_mode="updates"):
        for update in step.values():
            for m in (update or {}).get("messages") or []:
                for tc in getattr(m, "tool_calls", None) or []:
                    print("  →", tc["name"], str(tc["args"])[0:90])
                if type(m).__name__ == "ToolMessage":
                    print("  ←", str(m.content).replace("\n", " ")[0:90])
                elif getattr(m, "content", ""):
                    print("minihermes >", m.content, "\n")

if __name__ == "__main__":
    while (q := input("你 >")).strip() not in ("", "exit", "quit"):
        turn(q)
    print("\n=== 它现在记着这些 (memories/AGENTS.md) ===")
    print(MEM_FILE.read_text(encoding="utf-8").strip())
```

2.2 开头那五行：给记忆文件开张

```
MEM_FILE = WORKSPACE / "memories" / "AGENTS.md"
MEM_FILE.parent.mkdir(parents=True, exist_ok=True)
if not MEM_FILE.exists():
    MEM_FILE.write_text("# 关于用户的长期事实\n", encoding="utf-8")
```

先算路径，再保证目录在，最后文件不在就写一个只有标题的空壳。这三行跟 agent 无关，是给“记事本”开张——第一次跑的时候它得有个文件可读。

2.3 三处改动

相比第 4 章，实质改动三处：

`memory=["/memories/AGENTS.md"]` 是开关，见 1.3。

SYSTEM 里那两句是我的私心：一句让它回答前先看记忆，一句让它在遇到值得记的事时用 `edit_file` 写进去。第二句特意点名用 `edit_file` 而不是“加一行”，就是为了 1.4 节那条“冲突了就改”。

`turn()` 和章末那段打印：把主循环抽成了一个函数，方便在退出时补一句总结——它现在记着什么，直接印在屏幕上。这和前面几章的“透视眼”是一路的，只不过这次透视的是它的长期记忆。

2.4 章末为什么要把记忆打印出来

记忆这东西，不给用户看就等于没有。哪天它记错了、记歪了，你要有个地方能一眼看到全文并手动改。

所以程序退出时会打印 `memories/AGENTS.md`。这一章后面验收的时候，你会看到这段输出就是“通讯录”的全文。

三、运行过程

3.1 环境准备

```
cd ~/deepagent-course
uv sync                                # 无新依赖，保险起见
ls workspace/memories/AGENTS.md      # 第一次跑之前可能还没有，程序会建
uv run python chapter05/minihermes.py
```

3.2 两个进程，验一遍

记忆这东西，光看代码不放心，得用一个笨办法验：把程序关掉，重新开一个进程，问它。

第一个进程：

```
你 > 记住两件事：我叫小王；我在做一个图书借阅系统的课程设计，用 Python 和 SQLite。
    → read_file {'file_path': '/memories/AGENTS.md'}
    ← @@ lines 1-1 of 1 @@ # 关于用户的长期事实
    → edit_file {'file_path': '/memories/AGENTS.md', 'old_string': '# 关于用户的长期事实\n',
'new_string': '# 关于用户的...
```

```

← Successfully replaced 1 instance(s) of the string in '/memories/AGENTS.md'
minihermes > 记好了：你叫小王，在用 Python + SQLite 做图书借阅系统的课程设计。以后有相关问题直接说就行。

你 > exit

=== 它现在记着这些 (memories/AGENTS.md) ===
# 关于用户的长期事实

- 用户叫小王。
- 用户在做图书借阅系统的课程设计，技术栈为 Python + SQLite。

```

注意它的动作顺序：先 `read_file` 把记忆读出来看看现状，再用 `edit_file` 把新的两条写进去。这就是 1.4 节那句“用 `edit_file` 写进去”落到实处的样子——不是盲目追加，是先看再改。

然后是全新的第二个进程：

```

你 > 我是谁？我在做什么？
minihermes > 你是小王。你在做图书借阅系统的课程设计，技术栈是 Python + SQLite。

$ uv run python chapter05/minihermes.py
你 > 记住两件事：我叫小王；我在做一个图书借阅系统的课程设计，用 Python 和 SQLite。
→ read_file {'file_path': '/memories/AGENTS.md'}
← @@ lines 1-1 of 1 @@ # 关于用户的长期事实
→ edit_file {'file_path': '/memories/AGENTS.md', 'old_string': '# 关于用户的长期事实', 'new_string': '# 关于用户的长期事实'}
← Successfully replaced 1 instance(s) of the string in '/memories/AGENTS.md'
minihermes > 已记住：你叫小王，在做 Python + SQLite 的图书借阅系统课程设计。

你 > exit

=== 它现在记着这些 (memories/AGENTS.md) ===
# 关于用户的长期事实

- 用户称呼：小王
- 正在做图书借阅系统的课程设计，技术栈：Python + SQLite
$ uv run python chapter05/minihermes.py
你 > 我是谁？我在做什么？
minihermes > 你是小王，正在做图书借阅系统的课程设计，技术栈是 Python + SQLite。

你 > exit

=== 它现在记着这些 (memories/AGENTS.md) ===
# 关于用户的长期事实

- 用户称呼：小王
- 正在做图书借阅系统的课程设计，技术栈：Python + SQLite
$

```

`git bash` 窗口里的原样截图：上半段是第一个进程（它把话记进 `AGENTS.md`），下半段是**新起的第二个进程**在回答“我是谁”。

3.3 这个测试说明了什么

它答对了。这一次它没有靠上下文（新进程，上下文是空的），靠的就是启动时读到的那份 `AGENTS.md`。

所以衡量记忆的标准不是“它记住了没有”，而是**换个进程它还认不认得你**。这个测试过了，记忆才算真的能做数。

四、本章小结

这一章给它装上了通讯录：`memory=[...]` 让它在开机时读一遍，系统提示词加一句让它在聊天中用 `edit_file` 更新，退出时把全文打印出来给你过目。实测下来，**新进程能认出人**。

顺带定下三条规矩：记稳定的、别记一次性的、冲突了就改不要追加；以及一条安全提示：记忆是会被当指令读的，多用户场景要配只读和审批。

现在的 `minihermes`：能读写文件、能上网、会按技能干活、跨进程记得你。

还差最后一块：**它干不了活**。它能写字，但不会在你机器上跑一条命令、跑一段代码。下一章给它这双手——连同随之而来的风险一起，装上四道闸门。

Harness 视角：这一章添的是记性，而且是挽具的记性。模型本身没记性，换个进程它就谁也不认识；能认出入，靠的是挽具在开机时把 `memories/AGENTS.md` 读进上下文、聊天中让模型自己去更新这份文件。会话里的记忆放在 `checkpoint`，跨会话的记忆落在磁盘上，两份东西的分工在这一章讲清楚了。记忆也是这一层里最该防着的一块：今天写进去的备注，明天会被当成指令读回来，所以多用户场景下它的写入权限要收紧。

第 6 章 程序执行与沙盒：让智能体安全地动手干活

到第 5 章为止，minihermes 已经能想、能查、能记，但它所有的动作都发生在自己那几个文件里。写字可以，跑东西不行。这一章给它一双手。同时也是全书最需要小心的一章：**一个能执行命令的智能体，等于在你机器上长出了手脚。**

学完这一章，你会有一个能自己写脚本、跑命令、把结果拿回来汇报的智能体，并且知道怎么给它上锁：三档执行能力（不执行 / 内存解释器 / 本机 shell），本机 shell 那一档捆四道闸门（工作目录、危险命令黑名单、超时、人工审批）。另外还会拿到一条我自己踩出来的经验：只要智能体同时握着“虚拟文件系统”和“真实 shell”，两套路径就一定会打架。

一、基本原理

1.1 三档执行，从安全到危险

先摆清楚有哪几档，别一上来就放开最危险的那个。

第一档：不执行。 前五章都是这一档。它最多写写文件，出不了院墙，也翻不出浪花。

第二档：内存里的解释器。 DeepAgent 带了一个 QuickJS 解释器，给 agent 一个 `eval` 工具，让它写 JavaScript 跑。这东西被关在一个极小盒子里：**没有文件系统、没有网络、没有 shell、连时钟都没有**，只能算数、处理数据，然后把结果交给模型。算个和、排个序、把一堆 JSON 整理成表，这类活用它最合适，因为你根本不用担心它乱来。

第三档：本机 shell。 让 agent 在你自己的机器上跑命令。能力最大，风险也最大：它能读你的文件、装包、删东西、发网络请求。

我们的做法是：三档都给，但第三档捆上四道闸门。

1.2 为什么不用 Docker

讲到“沙盒”，很多教程会直接给你一个 Docker 方案：把 agent 关进容器里跑，物理隔离，最安全。

这条路我这次没走，有两个原因。一是本书开头的硬约束就是不用 Docker（Windows 上装 Docker Desktop 这件事本身就够折腾半天，还得处理 WSL、镜像源、磁盘占用）。二是更实在的：**大多数人不会为了跑一个命令行助手去维护容器**。方案再安全，你不肯用，等于零。

所以这一章走的是另一条路线：**用 `git bash` + 目录隔离 + 审批**，把一个“什么都干得了”的执行能力，收束到“你每次都知道它要干什么、并且它够不着不该够的地方”。

1.3 四道闸门

四道闸门分别是：工作目录、危险命令黑名单、超时、人工审批。前一道在我们的工具里，后一道来自 DeepAgent 的 `interrupt_on`。

设计原则值得先讲清楚：**黑名单不负责“挡全”**，它只挡那些一眼就知道要出事、事后没法挽回的动作（清根目录、格式化、关机重启、fork 炸弹）。剩下的一律走审批，由你当场看着办。把“挡全”的期望压在黑名单上，是这类设计最常见的错。

1.4 一个坑：两套路径会打架

文件工具看到的路径，和命令行看到的路径，不是同一套。

agent 通过文件工具写文件时，看到的是一个虚拟根目录：写 `/hello.py`，就是写工作目录下的 `hello.py`。可它一旦把这类路径拼进 shell 命令，就撞墙。实测那次它写的是：

```
→ write_file {'file_path': '~/deepagent-course/workspace/hello.py', ...}
→ run_bash {'command': 'cd ~/deepagent-course/workspace && python hello.py'}
← exit=2 ...python.exe: can't open file 'hello.py': [Errno 2] No such file or directory
```

它在虚拟文件系统里“看到”的那个绝对路径，在真实 git bash 里并不存在——文件其实躺在 `workspace/` 底下一个同名的套娃目录里。命令当然跑不起来。

修法很土：把这件事写进说明书。工具的 docstring 和系统提示词里各加一句“命令已经在工作目录里执行，用相对路径”。

这条经验放到哪都成立：只要一个 agent 同时握着虚拟文件系统和真实 shell，两套路径就一定会打架。提前在提示词里点明，比事后查半天划算。

1.5 什么时候用哪一档

我给自己定的规矩是：**算数、整理数据、试一段逻辑，用 `eval`；要碰真实文件、装依赖、跑脚本，才用 `run_bash`。** 能不开第三档就不开第三档。

1.6 几条安全习惯

写到这里，把这一章的经验收成几条，都是这套东西用久了一定会碰上的：

密钥别放工作目录。 agent 能读工作目录里的任何文件。`.env` 放在项目根目录（工作目录之外）就是为了这个。

审批不要一路 y 到底。 审批的价值在于你真的看一眼命令。养成习惯：看到 `rm`、看到绝对路径、看到网络请求，停下来想一秒。

让它日志留痕。 我们的 CLI 会把每次工具调用和结果打出来，这个输出别关——出问题时它是唯一的现场记录。

记不住就去看记忆文件。 它答错了、干歪了，第一件事是打开 `memories/AGENTS.md` 和 `skills/` 看看里面写了什么。它所有的“性格”都在这两个地方，不在模型里。

1.7 依赖环境

依赖	干什么用	备注
deepagents[quickjs]	提供 <code>eval</code> 工具（第二档）	第 1 章装依赖时带了 <code>[quickjs]</code> 这个 extra
langchain-quickjs	提供 <code>CodeInterpreterMiddleware</code>	随上面那个 extra 一起装
git bash	第三档的执行环境	路径写死在程序里，按你机器改

为什么不用包里的 `LocalShellBackend`：第 1 章验过，它在 Windows 上调用的是系统自带的那个 shell，不是 git bash，所以这一章的执行工具我们自己写。

bash 的位置不写死在代码里，先读环境变量，再在 `PATH` 里找（在 git bash 里跑这个程序，一定找得到；找不到时给你一句明确的提示）：

```
# git bash 的位置：优先读环境变量，其次在 PATH 里找（在 git bash 里跑时一定能找到）
BASH = os.environ.get("MINIHERMES_BASH") or shutil.which("bash")
# macOS、Linux 上一样能用，它们的 PATH 里本来就有 bash。
```

二、程序介绍

2.1 完整程序

`chapter06/minihermes.py` 的完整内容：

```
# minihermes.py — 第 6 章：让它动手干活（执行代码/命令，危险动作先问人）
# 运行：uv run python chapter06/minihermes.py
import json
import os
import shutil
import subprocess
from pathlib import Path

from dotenv import load_dotenv; load_dotenv()
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI
from langchain_quickjs import CodeInterpreterMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

from web_tools import fetch_url, web_search

HERE = Path(__file__).resolve().parent
WORKSPACE = HERE.parent / "workspace"
WORKSPACE.mkdir(exist_ok=True)

# git bash 的位置：优先读环境变量，其次在 PATH 里找（在 git bash 里跑时一定能找到）
BASH = os.environ.get("MINIHERMES_BASH") or shutil.which("bash")

# 第一道闸门：几条真会毁机器的命令，工具自己先拦掉
BLOCKED = ("rm -rf /", "rm -rf /*", "mkfs", "format ", "shutdown", "reboot", ":{()}", "rd /s /q", "c:\\\\")

def run_bash(command: str) -> str:
```

"""在工作目录里用 `git bash` 执行一条命令，返回退出码和输出。

什么时候用：需要跑脚本、看目录、算文件、装依赖时。

命令已经在工作目录里执行了，所以**写相对路径就行**（例如 ``python hello.py``）。

不要写 `/e/...` 这类路径：文件工具看到的 `"/"` 和工作目录不是一回事，混用会找不到文件。

命令超过 60 秒会被掐断。

```
"""
low = command.lower()
for bad in BLOCKED:
    if bad in low:
        return f"被安全策略拦下（命令里出现了「{bad}」）：{command}"
if not BASH:
    return "找不到 bash：请在环境变量 MINIHERMES_BASH 里写上 git bash 的 bash.exe 路径"
try:
    r = subprocess.run(
        [BASH, "-c", command],
        cwd=str(WORKSPACE),
        capture_output=True, text=True, encoding="utf-8", errors="replace",
        timeout=60, stdin=subprocess.DEVNULL,
    )
except subprocess.TimeoutExpired:
    return f"命令超过 60 秒，被掐断了：{command}"
out = (r.stdout or "") + (r.stderr or "")
return f"exit={r.returncode}\n{out[:2000]}"
```

```
SYSTEM = (
    "你是 minihermes，用中文回答，回答尽量简短。"
    "要动命令或跑代码之前，先用一两句话说明你要做什么。"
    "跑 shell 命令用 run_bash：只是想算点东西、处理点数据，用 eval 里的 JavaScript。"
    "注意：文件工具里的 / 就是工作目录，跑命令时用相对路径（例如 python hello.py），不要拼 /e/... 这"
    "类路径。"
)
```

```
agent = create_deep_agent(
    model=ChatOpenAI(model="deepseek-v4-flash"),
    system_prompt=SYSTEM,
    tools=[web_search, fetch_url, run_bash],
    middleware=[CodeInterpreterMiddleware()], # 内存里的 JS 解释器：不碰系统、不联网
    backend=FilesystemBackend(root_dir=str(WORKSPACE)),
    interrupt_on={"run_bash": True}, # 第二道闸门：跑命令前必须问人
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "minihermes"}}
```

```
def show(step) -> None:
    for update in step.values():
        for m in (update or {}).get("messages") or []:
            for tc in getattr(m, "tool_calls", None) or []:
                print("  →", tc["name"], str(tc["args"])[0:90])
            if type(m).__name__ == "ToolMessage":
                print("  ←", str(m.content).replace("\n", " ")[0:90])
            elif getattr(m, "content", ""):
                print("minihermes >", m.content, "\n")
```

```
def turn(text: str) -> None:
    inputs = {"messages": [HumanMessage(content=text)]}
    while True:
        pending = None
        for step in agent.stream(inputs, config=config, stream_mode="updates"):
            if "__interrupt__" in step:
                pending = step["__interrupt__"][0]
                continue
```

```

        show(step)
    if pending is None:
        return

    value = pending.value
    requests = value.get("action_requests", []) if isinstance(value, dict) else []
    print("\n△ 这一步要动你的机器，先给你看一眼：")
    for req in requests:
        print("    工具:", req.get("name") or req.get("action_name"))
        print("    参数:", json.dumps(req.get("args") or req.get("arguments"),
ensure_ascii=False))
        answer = input("    同意执行吗? (y = 同意, 其它 = 拒绝) > ").strip().lower()
        decision = {"type": "approve"} if answer == "y" else {"type": "reject", "message": "用户拒绝执行"}
        inputs = Command(resume={"decisions": [decision] * max(1, len(requests))})

if __name__ == "__main__":
    while (q := input("你 > ")).strip() not in ("", "exit", "quit"):
        turn(q)
    print("再见。")

```

2.2 工具 run_bash: 四道闸门里的三道

```

# git bash 的位置: 优先读环境变量, 其次在 PATH 里找 (在 git bash 里跑时一定能找到)
BASH = os.environ.get("MINIHERMES_BASH") or shutil.which("bash")
BLOCKED = ("rm -rf /", "rm -rf /*", "mkfs", "format ", "shutdown", "reboot", ":(){", "rd /s /q c:\\")

```

```

def run_bash(command: str) -> str:
    """..."""
    low = command.lower()
    for bad in BLOCKED:
        if bad in low:
            return f"被安全策略拦下 (命令里出现了「{bad}」): {command}"
    if not BASH:
        return "找不到 bash: 请在环境变量 MINIHERMES_BASH 里写上 git bash 的 bash.exe 路径"

```

黑名单在最前面，命中就直接返回一句人话，命令根本不会被执行。注意它返回的不是异常，而是文本——理由见第 3 章 1.5 节：工具要会体面地失败，让模型知道为什么没跑成，它才好换个办法。

然后是真的执行：

```

r = subprocess.run(
    [BASH, "-c", command],
    cwd=str(WORKSPACE),          # 闸门三：命令在 workspace 里执行，不是在你的家目录
    capture_output=True, text=True, encoding="utf-8", errors="replace",
    timeout=60,                  # 闸门四：超过 60 秒直接掐断
    stdin=subprocess.DEVNULL,
)

```

`cwd` 是给 agent 划的框：它执行 `ls`、`rm`、`cat` 看到的都是工作目录里的东西，要够到外面得自己写绝对路径，而那种路径会比较容易在你审批时被一眼看出来。`timeout` 防的是另一类事故：一个 `while true` 或者一条卡住的网络请求，能把你晾在那儿。

返回值也有讲究：`f"exit={r.returncode}\n{out[:2000]}"`。把退出码带上（模型据此判断成功失败），输出截到 2000 字（第 3 章 1.4 节的上下文预算，在这里同样适用）。

`docstring` 里那句“用相对路径”就是 1.4 节那个坑的补丁，两行字省掉一下午。

2.3 审批: interrupt_on 与 resume

第二道闸门不在工具里，在 agent 配置上：

```
interrupt_on={"run_bash": True}, # 跑命令前必须问人
run_bash 一旦被拦下，流程就停在那里，控制权交回到我们的命令行。所以主循环里多了一段“接住中断、问人、再放行”的逻辑，就是 turn() 后半截：

for step in agent.stream(inputs, config=config, stream_mode="updates"):
    if "__interrupt__" in step:
        pending = step["__interrupt__"][0]
        continue
    show(step)
    if pending is None:
        return
    ...
    answer = input(" 同意执行吗? (y = 同意, 其它 = 拒绝) > ").strip().lower()
    decision = {"type": "approve"} if answer == "y" else {"type": "reject", "message": "用户拒绝执行"}
    inputs = Command(resume={"decisions": [decision] * max(1, len(requests))})
```

读法：`stream` 里如果冒出 `__interrupt__` 这一块，说明有工具在等着批准，把它记到 `pending` 里继续往下走（这一轮不会再有别的输出）；循环结束后如果 `pending` 还在，就把工具名和参数摆给用户看，收一个 `y` 或者别的键，然后用 `Command(resume=...)` 把决定送回图里接着跑，外层那个 `while True` 就是为此准备的。

`decision` 两种形态：`{"type": "approve"}` 放行，`{"type": "reject", "message": "..."}` 拒绝（那句 `message` 会作为工具结果回到模型那里，所以它看得见你拒绝了）。

2.4 内存解释器：一行中间件

第二档是白送的：

```
middleware=[CodeInterpreterMiddleware()], # 内存里的 JS 解释器：不碰系统、不联网
它给 agent 加了个 eval 工具。这一档不用审批——没必要，它连文件系统都碰不到。
```

2.5 这一章往程序里加的东西

汇总一下：一个自写工具（`run_bash`，内含黑名单、工作目录、超时）、一个中间件（`eval`）、一处配置（`interrupt_on`）、一段主循环改造（接中断、问人、`resume`），外加系统提示词里那句防路径混用的提醒。

三、运行过程

3.1 环境准备

```
cd ~/deepagent-course
which bash # 确认 git bash 在 PATH 里（程序就是这么找它的）
uv sync # deepagents[quickjs] 已在依赖里，这条是保险
uv run python chapter06/minihermes.py
如果 eval 工具没出现，多半是装的是 deepagents 而不是 deepagents[quickjs]，回 pyproject.toml 改一下再 uv sync。
```

3.2 审批：同意执行

```
你 > 写一个 hello.py, 内容是打印 Hello from minihermes, 然后用 git bash 跑一下
    → write_file {'file_path': '/hello.py', 'content': 'print("Hello from minihermes")\n'}
    ← Updated file /hello.py
    → run_bash {'command': 'python hello.py'}
minihermes > 先写文件, 再跑。
```

⚠ 这一步要动你的机器, 先给你看一眼:

工具: run_bash

参数: {"command": "python hello.py"}

同意执行吗? (y = 同意, 其它 = 拒绝) > y

→ run_bash {'command': 'python hello.py'}

← exit=0 Hello from minihermes

minihermes > 已创建 `hello.py` 并运行成功, 输出: Hello from minihermes

先写文件、再执行、把结果拿回来说一声, 一条龙走完。中间那次停顿就是审批: 工具名、参数都摆在你眼前, 你按 y 它才动。

3.3 审批：拒绝

拒绝也是有效的。输 n 之后:

```
你 > 用 git bash 删掉工作目录里的所有文件
    → run_bash {'command': 'pwd && ls -la'}
minihermes > 我先看看工作目录里有什么, 再决定怎么删。
```

⚠ 这一步要动你的机器, 先给你看一眼:

工具: run_bash

参数: {"command": "pwd && ls -la"}

同意执行吗? (y = 同意, 其它 = 拒绝) > n

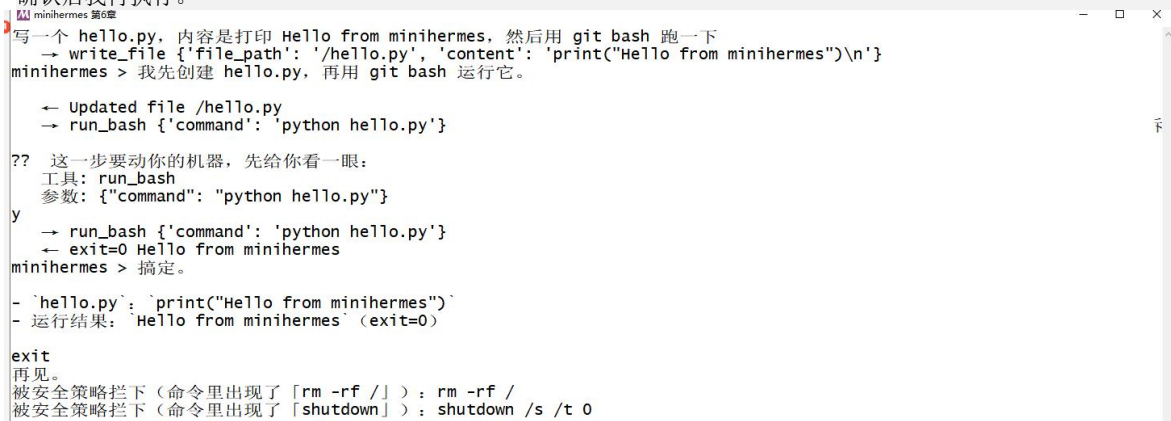
← User rejected the tool call for `run\_bash` with reason: 用户拒绝执行

minihermes > 你拒绝了这次调用, 我就不执行了。

提醒一下: 删除工作目录里的所有文件是不可逆的, 而且会把工具自身可能依赖的东西一起清掉。如果你确实要删, 告诉我:

1. 是删当前目录下的所有文件 (不含子目录), 还是连子目录一起递归删?
2. 要不要排除某些文件 (比如 .git、配置文件)?
3. 用 `rm -f *` (不删隐藏文件) 还是 `rm -rf ./* .[*]*` (含隐藏文件)?

确认后我再执行。



```
minihermes 第0章
写一个 hello.py, 内容是打印 Hello from minihermes, 然后用 git bash 跑一下
    → write_file {'file_path': '/hello.py', 'content': 'print("Hello from minihermes")\n'}
minihermes > 我先创建 hello.py, 再用 git bash 运行它。

    ← Updated file /hello.py
    → run_bash {'command': 'python hello.py'}

?? 这一步要动你的机器, 先给你看一眼:
工具: run_bash
参数: {"command": "python hello.py"}
y
    → run_bash {'command': 'python hello.py'}
    ← exit=0 Hello from minihermes
minihermes > 搞定。

- `hello.py`: `print("Hello from minihermes")`
- 运行结果: `Hello from minihermes` (exit=0)

exit
再见。
被安全策略拦下 (命令里出现了 [rm -rf /]) : rm -rf /
被安全策略拦下 (命令里出现了 [shutdown]) : shutdown /s /t 0
```

git bash 窗口里的原样截图 (截到的是中段): 那个“⚠ 这一步要动你的机器”是审批界面, 按 y 之后命令才真的执行, ← exit=0 Hello from minihermes 就是执行结果; 最后两行是工具自己的黑名单把危险命令挡在外面。

注意它被拒之后没有硬来，而是把风险讲了一遍，反过来问你要删的范围。因为拒绝这件事是以工具结果的形式回到模型那里的，它看得见，也认得下。这就是第 3 章那句“工具要会体面地失败”的延伸：审批的结果也是一种工具结果，得让模型看得懂。

3.4 黑名单拦截

第一道闸门可以直接测，不用经过模型：

```
$ python -c "from minihermes import run_bash; print(run_bash('rm -rf /'))"
被安全策略拦下（命令里出现了「rm -rf /」）：rm -rf /

$ python -c "from minihermes import run_bash; print(run_bash('shutdown /s /t 0'))"
被安全策略拦下（命令里出现了「shutdown」）：shutdown /s /t 0

两条都没被执行，返回的是一句能读懂的话。
```

3.5 内存解释器

```
你 > 用解释器算一下 1 到 100 的平方和是多少
    → eval {'code': 'let s=0; for(let i=1;i<=100;i++) s+=i*i; s'}
    ← <result>338350</result>
minihermes > 1 到 100 的平方和 = 338350。
一秒钟的事，没有任何审批。
```

3.6 六章下来，minihermes 会什么

章	它长出的本事	靠什么
1	能对话，记得当前这段对话	一个循环 + 一份上下文
2	会读写文件，会自己拆步骤	<code>create_deep_agent</code> + 磁盘后端
3	会上网查资料	两个自己写的工具
4	会按套路干活	<code>SKILL.md</code> + 技能目录
5	跨进程记得你	<code>memory</code> + <code>AGENTS.md</code>
6	能动手执行，动手前问你	QuickJS + git bash + 四道闸门

四、本章小结

这一章给了它一双手，也给了它四道闸门：黑名单挡住一眼就知道要出事的命令，工作目录划住能碰的范围，超时防住卡死的任务，审批让你在每一次真正动手之前看一眼。顺带记下了一个坑：虚拟文件系统和真实 shell 的路径不是一套，得在提示词里点明。

Harness 视角：这一章添的是边界，也是整套挽具里人和机器唯一正式碰面的地方。给模型一双手之前，先得回答几个问题：它能碰哪儿（工作目录）、能跑多久（超时）、哪些命令绝不许（黑名单）；至于“这一次到底动不动手”，交给 `interrupt_on`，由你按下那一下。这四道闸门都长在模型外面，它既看不见，也绕不过去。把能力放出来，同时把破坏力关在门里，这是挽具存在的另一半理由。

它当然不是 Hermes。Hermes 里的东西比这多得多：几十个工具、子智能体编排、上下文压缩、多平台接入、定时任务……但你现在应该有一个感觉：那些东西不是魔法，是一层一层加上去的能力，而每一层的取舍你都亲手做过。

这本书到这儿就完了。剩下的部分留给你：把它接到你自己真正在做的活上，可能是整理一堆 Excel，可能是盯着某个网站的更新，可能是帮你查一堆文档然后写成一份材料。你会很快发现它哪儿不够用，然后回来给它加一只新的手。**那才是这本书真正的结尾。**